Information Fusion 123 (2025) 103272

Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus

Full length article

IncA-DES: An incremental and adaptive dynamic ensemble selection approach using online K-d tree neighborhood search for data streams with concept drift

Eduardo V.L. Barboza [a,b,*], Paulo R. Lisboa de Almeida [b], Alceu de Souza Britto Jr. [c], Robert Sabourin [a], Rafael M.O. Cruz [a]

[a] LIVIA, École de Technologie Supérieure, University of Québec, Montreal, Québec, Canada
[b] Department of Informatics, Universidade Federal do Paraná (UFPR), Curitiba (PR), Brazil
[c] Graduate Program in Informatics (PPGIa), Pontifícia Universidade Católica do Paraná (PUCPR), Curitiba (PR), Brazil

ARTICLE INFO

Keywords:
Data streams
Concept drift
Machine learning
Dynamic selection
K-d tree

ABSTRACT

Data streams pose challenges not usually encountered in batch-based Machine Learning (ML). One of them is concept drift, which is characterized by the change in data distribution over time. Among many approaches explored in literature, the fusion of classifiers has been showing good results and is getting growing attention. More specifically, Dynamic Selection (DS) methods, due to the ensemble being instance-based, seem to be a more efficient choice under drifting scenarios. However, some attention must be paid to adapting such methods for concept drift. The training must be done in order to create local experts, and the commonly used neighborhood-search DS may become prohibitive with the continuous arrival of data. In this work, we propose Incremental Adaptive Dynamic Ensemble Selection (IncA-DES), which employs a training strategy that promotes the generation of local experts with the assumption that different regions of the feature space become available with time. Additionally, the fusion of a concept drift detector supports the maintenance of information and adaptation to a new concept. An overlap-based classification filter is also employed in order to avoid using the DS method when there is a consensus in the neighborhood, a strategy that we argue every DS method should employ, as it was shown to make them more applicable and quicker. Moreover, aiming to reduce the processing time of the k-Nearest Neighbors (kNN), we propose an Online K-d tree algorithm, which can quickly remove instances without becoming inconsistent and deals with unbalancing concerns that may occur in data streams. Experimental results showed that the proposed framework got the best average accuracy compared to seven state-of-the-art methods considering different levels of label availability and presented the smaller processing time between the most accurate methods. Additionally, the fusion with the Online K-d tree has improved processing time with a negligible loss in accuracy. We have made our framework available in an online repository[1].

1. Introduction

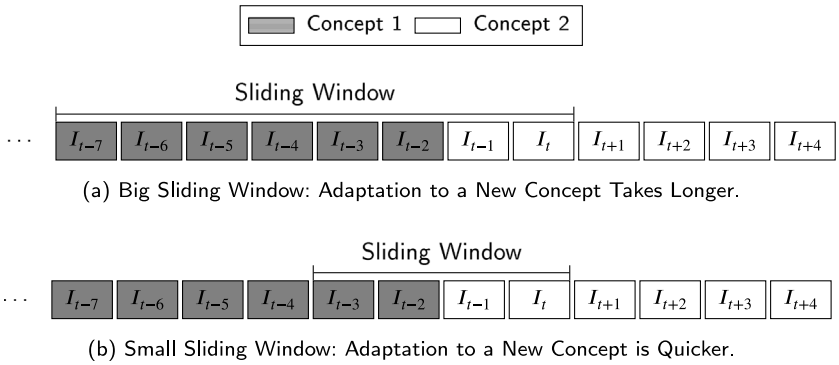
When dealing with data streams, where both training and testing instances arrive over time, we may encounter some scenarios usually not found in classic batch-based Machine Learning (ML). One of them is concept drift, where data distribution changes over time [1,2]. Under such scenario, the ML models must constantly learn with the newly arrived data and adapt to changing distributions, which may involve discard old and possibly conflicting information and/or to learn on the newest data. Concept drift may be encountered in, for instance, cybersecurity-related problems [3,4], climate prediction [5], spam prediction [6,7], and people's opinion [8].

In literature, there are two main approaches to dealing with concept drift. In the first one, called the blind adaptation strategy, the classification models are kept up to date only with the most recent data, discarding past samples unaware of concept drift [9,10]. The second strategy, active strategy, tries to track concept drift, primarily by monitoring the classification error or perceiving changes in the data distribution through concept drift detectors, also known as triggers [11,12].

* Corresponding author at: LIVIA, École de Technologie Supérieure, University of Québec, Montreal, Québec, Canada.
E-mail address: eduardo.lima-barboza.1@ens.etsmtl.ca (E.V.L. Barboza).

[1] https://github.com/eduardovlb/IncA-DES

https://doi.org/10.1016/j.inffus.2025.103272
Received 16 December 2024; Received in revised form 22 April 2025; Accepted 23 April 2025
Available online 8 May 2025



(a) Big Sliding Window: Adaptation to a New Concept Takes Longer.

(b) Small Sliding Window: Adaptation to a New Concept is Quicker.

Fig. 1. The Impact of Window Size on Adapting to Concept Drift.

Recently, Dynamic Selection (DS)-based approaches have been getting growing attention for concept drift scenarios [13–15], as they can select the most promising ensemble of classifiers based on the characteristics of the test instance. In many methods, the most competent candidate classifiers in the Region of Competence (RoC) of the test instance, commonly gathered through a k-Nearest Neighbors (kNN) algorithm from the named Dynamic Selection Set (DSEL), are chosen according to their performance. The DSEL is the set of labeled instances used by the DS method for selecting the classifiers. Therefore, the candidate classifiers from the pool C that are more competent in the RoC are chosen to classify the test instance, and local experts of the RoC are more likely to be chosen if they exist.

Given the possibility of concept drift, many DS-based approaches rely on blind adaptation strategies, keeping a sliding window of recent instances or data chunks as a DSEL [13,16,17], making DS both space and time-dependent. Many of them employ the fusion of concept drift detectors as well [15,17,18] to drop old instances from the DSEL or to update the pool of candidate classifiers. To differ the sliding window employed in DS for data streams to the DSEL used in static DS, we call it Dynamic Selection Window (DSEW).

However, the DSEW may suffer from the well-known stability-plasticity dilemma problem [19] since, on the one hand, a big DSEW will have more information for the DS mechanism, but adapting to a new concept may take longer. On the other hand, a smaller DSEW may lead to a quicker adaption but results in a lack of information to estimate the RoC. As exemplified in Fig. 1, as the window slides through the new arriving instances, the old concept remains until the new concept has enough instances to overlap it completely. The larger the sliding window, the longer the adaptation will take. This can even become a bigger problem under a limited label availability that happens in real-world scenarios, mainly when human action is needed to label the data. Thus, leveraging helpful information while aware of possible changes is crucial in drifting scenarios.

Further, the training approach that DS methods for data streams is usually employ is based on some resampling strategy, such as online bagging [13,15]. However, such an approach may not be the best one for DS. It has a global perspective of the data and does not guarantee the creation of local experts, preferred for ensemble candidates in DS [20]. Cruz et al. [20] also mentions that techniques leveraged from static ML may not be the best choices for training models for Dynamic Ensemble Selection (DES) applications.

In this work, we propose Incremental Adaptive Dynamic Ensemble Selection (IncA-DES), which adopts an incremental training approach that favors local experts' generation as the feature space regions become available over time. Besides, considering a limited label availability, focusing the training on only one classifier may bring advantages compared to resampling, as the latter may cause classifiers not to receive enough information, leading to a pool of underfitted candidate classifiers.

In addition, concept drift is addressed through drift detection. The idea is to have a DSEW that can keep as many instances as possible. When a drift detector triggers a concept drift, the DSEW shrinks to the warning level given by the drift detector to adapt to the new concept. Thus, we can retain useful information and drop potentially outdated instances when a concept drift is detected. The rationale is to let the drift detector be in charge of dealing with *real concept drift*, while under stable concepts or *virtual concept drift*, information is leveraged for DS. Furthermore, faced with the possibility of false alarms from the drift detector, old classifiers are still kept as ensemble candidates, making IncA-DES robust in these scenarios.

To enhance the effectiveness of the proposed framework, we present the overlap-based classification, which can choose whether it is necessary to use the DS method for classifying a test instance. As we use a neighborhood-based DS method, and the kNN suffers more on classifying regions with a higher instance hardness [21], we use the DS method when the neighbors' classes are different. Otherwise, the kNN is used. Experimental analysis has shown that this mechanism has substantially decreased the processing time of the framework, while improving accuracy performance. Thus, we argue that every neighborhood-based DS method should employ this strategy.

Still into the kNN algorithm, many DS methods employ it to define the RoC to measure the competence of the classifiers [13,16]. A brute force kNN can be prohibitive under a streaming scenario, where decisions must be made in real-time due to the fast and possibly infinite number of instances arriving [22]. To address this, we propose an Online K-d tree algorithm for approximate neighborhood search. It can quickly process arriving instances and mitigate some problems found in the original K-d tree, such as deletion complexity, unbalancing with the addition of new nodes, and the need for normalized data. The presented Online K-d tree algorithm has promoted extensibility in the proposed framework, with a negligible loss in accuracy performance, which is expected in approximate neighborhood search. Fig. 2 shows an overview of the proposed framework. Notice that the classifiers are trained in different distributions of the data as they arrive over time.

Experimental analysis carried out on 22 datasets, including 11 real-world, 4 with an induced *virtual concept drift*, and 7 synthetic datasets, has shown that the proposed IncA-DES presented better accuracy performance in comparison with state-of-the-art methods, including DS and ensemble methods for concept drift, in scenarios with different levels of label availability. Further, the fusion of the framework with our proposed Online K-d tree algorithm reduced the overall processing time compared to the brute force kNN, with a negligible loss in accuracy. The overlap-based classification filter has enhanced effectiveness, bringing gain in both accuracy and speed.

Our main contributions are as follows:

- We propose a novel framework based on DS with a training policy that favors the generation of local experts and uses a drift detector to have an Adaptive DSEW.

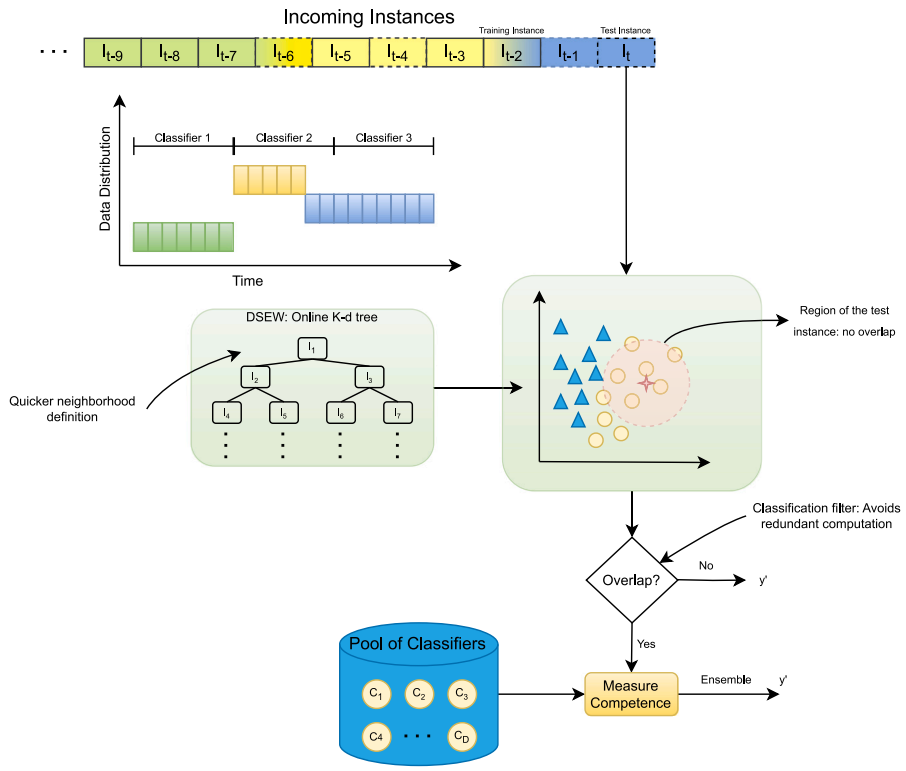


Fig. 2. An overview of IncA-DES and its components. Notice that the incremental training policy aims to train classifiers in different perspectives of data and the local region has no class overlap.

- We optimize the neighborhood search with an Online K-d Tree algorithm that can quickly process incoming instances in data streams.
- We apply a local overlap-based classification optimization of DS, which has promoted effectiveness in the framework by improving the accuracy performance and speeding up the processing of instances.
- We perform experiments considering label availability, which better reflects environments found in the real world, comparing several DS and ensemble methods for data streams with concept drift.

The remainder of this work is structured as follows: in Section 2, we define key concepts used in this work, such as concept drift, batch processing, online processing, and basic concepts from DS. We present the proposed framework in Section 3, in Section 4, we present related works in the literature on concept drift detection and ensemble learning for concept drift, perform experiments comparing with state of the art and an ablation study with the presented components in Section 5, and conclude the paper in Section 6.

2. Theoretical foundation

2.1. Concept drift

In this work, we consider that a concept drift can be *real* or *virtual*, shown in Fig. 3. Given that $P_t(y|\mathbf{x})$ is the *a posteriori* probability relating a feature vector $\mathbf{x}$ with the target class y at a given time t, a *real concept drift* happens when $P_t(y|\mathbf{x}) \neq P_{t+\delta}(y|\mathbf{x})$, for any $\delta > 0$ [1,2,16,23,24]. In a *real concept drift*, the relation between the feature vectors and the target class changes over time, influencing the ideal decision boundary, as shown in Fig. 3(b), often bringing the need to retrain models from scratch.

A *virtual concept drift*, on the other hand, does not affect the ideal decision boundaries, as in Fig. 3(c). It is also described as a stable concept where different regions of the feature space become available over time [25], meaning that old data is still relevant. It happens when there is a change in the unconditional or the *a priori* distributions between t and $t+\delta$, while the *a posteriori* probability remains the same.

A *virtual concept drift* is given by $P_t(\mathbf{x}) \neq P_{t+\delta}(\mathbf{x})$ and/or $P_t(y) \neq P_{t+\delta}(y)$, while $P_t(y|\mathbf{x}) = P_{t+\delta}(y|\mathbf{x})$ [1,2,23]. Although *virtual concept drift* does not directly change the decision boundary, it is often the case that the models need to be updated, as the decision boundaries learned in the past may not reflect the current distribution, or have a limited view of it.

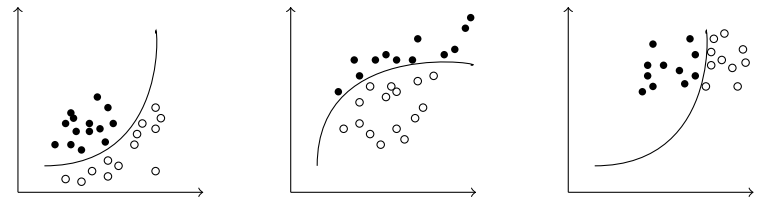
From an image recognition perspective, the arrival of winter may change the appearance of the space after it snows, but not necessarily invalidate the data learned in spring, summer, or fall, characterizing a *virtual concept drift*. Furthermore, when we started to use masks during the COVID-19 pandemic, face recognition models had to learn the new concept of people wearing masks, but maskless faces are still faces [26].

In summary, under *real concept drift* we usually need to update decision models to learn the new decision boundary, as part, or the whole information from the past may be conflicting with the current environment. In *virtual concept drift*, old information is still relevant. Thus, the best strategy in this case is to integrate new data into the information already known from the past. In other words, if a classifier knew in advance the ideal decision boundary for the problem at time t, there would be no need to do anything at $t + \delta$ under a *virtual concept drift*, while under a *real concept drift* this boundary would be invalidated.

There are still many other concept drift properties in the literature that we do not describe in detail in this work, such as the concept drift severity, speed, and recurrence. For a more in-depth discussion regarding different types of concept drift, refer to [2,23,24,27–29].

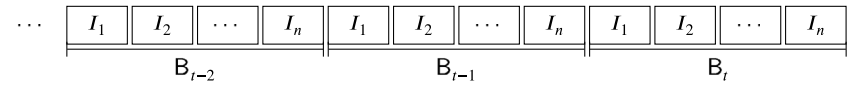
2.2. Batch versus online processing

The concept drift literature often assume that the data used to update the models is made available in batches [10,12,16] or single

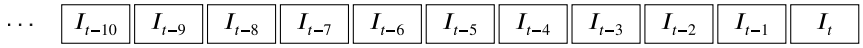


(a) Original Data at time t. (b) Real Concept Drift at time $t + \delta$. (c) Virtual Concept Drift at time $t + \delta$.

Fig. 3. Types of Concept Drift — Probabilistic source. Notice that *real concept drift* changes the best decision boundary, while *virtual concept drift* does not.



(a) Stream of batches. Each batch contains n instances.



(b) Stream of individual instances.

Fig. 4. Stream of instances vs stream of batches.

instances [9,30], shown in Fig. 4. Batch processing methods (Fig. 4(a)) assume that the labeled data used to update the models comes in chunks of a fixed size, containing $n > 1$ instances (e.g., we may receive a new batch of labeled instances every month). In contrast, single instance-based, i.e., online processing methods (Fig. 4(b)), update their models with individual instances. In this work, we consider the online processing approach as it can readily process an instance when it is available.

2.3. Dynamic selection

In this section, we describe concepts regarding DS. Firstly, let us define some key notations of DS techniques that will be used throughout this paper:

- $C = \{c_1, \ldots c_D\}$: Pool containing D classifiers.
- $\mathbf{x}$: an unlabeled test instance.
- $I = \{\mathbf{x}, y\}$: a labeled instance containing a feature vector $\mathbf{x}$ and the label y.
- DSEL: The validation set with labeled instances that are used to measure the competence of the classifiers.
- $\theta = \{I_1, \ldots, I_k\}$: Region of Competence (RoC) of the test instance $\mathbf{x}$, such that $\theta \subset DSEL$.

DS techniques are multiple classifier systems that build the ensemble on the testing phase, depending on the characteristics of the test instance, and usually follow three phases: pool generation, selection and integration [31]. By having a pool of classifiers, the goal is to select the most promising ones in the test instance and integrate them as an ensemble to classify the test instance. Depending on the DS method, these steps may change. For example, in Dynamic Classifier Selection (DCS) methods, the single best classifier will classify the test instance. In such a case, the integration part is not needed.

The goal of the pool generation phase is to generate an accurate and diverse pool of candidate classifiers C [31]. In the case of DS for data streams, the online bagging technique [32] is commonly used, in which all of the classifiers in C have a chance to be trained on every incoming instance. However, by taking into account the need to have a diverse pool of candidate classifiers, there may be better approaches than the online bagging to generate C. Considering that the local regions of the feature space become available over time, we use an incremental training approach for training the classifiers in C in our proposed framework.

The selection and integration phases in DS can be merged into the classification phase, which usually presents three steps [20]:

1. θ definition: is the RoC, commonly defined through a kNN algorithm;
2. Selection Criteria (Accuracy, Ranking, etc.): dictates how the system will evaluate which classifier(s) will be used to classify the test instance;
3. Selection Mechanism (Dynamic Classifier Selection (DCS) or Dynamic Ensemble Selection (DES)): determines whether only the single best classifier will label the test instance or if an ensemble of the classifiers that meet the selection criteria will be built.

The θ is where the competence of candidate classifiers is measured according to the selection criteria. It is commonly defined by using a kNN algorithm and is denoted by $\theta = \{I_1, \ldots, I_k\}$, where k is the number of neighbors, and I_k is a labeled instance. To do so, we need a set of labeled instances named DSEL, which can be either the training or a validation set. Besides kNN, examples of other approaches explored in literature for DS are fuzzy hyperbox [33], Graph Neural Networks [34], and meta-features [35].

DS techniques that run under data streams often use a sliding window on the incoming labeled instances as the DSEL. In this work, when referring to such validation set for DS, we will use the term DSEW to differ from the DSEL used in static DS.

After the set θ is defined, the competence of the candidate classifiers is measured based on the selection criteria. DCS methods select the single best classifier, while DES may select one or more classifiers to compose an ensemble. For instance, a DES method could select all classifiers that correctly classify all instances in θ. Then, the chosen classifiers are integrated as an ensemble to classify $\mathbf{x}$.

3. Proposed method — Incremental Adaptive Dynamic Ensemble Selection (IncA-DES)

In this work, we propose IncA-DES, a DS-based framework for data streams that address concept drift. It can leverage information of stable concepts and adapt to concept drift when needed. Fig. 5 shows an overview of the proposed framework when a newly labeled training sample is available and when a sample comes for being classified. Notice that the *trigger* component is responsible for adapting to concept drift.

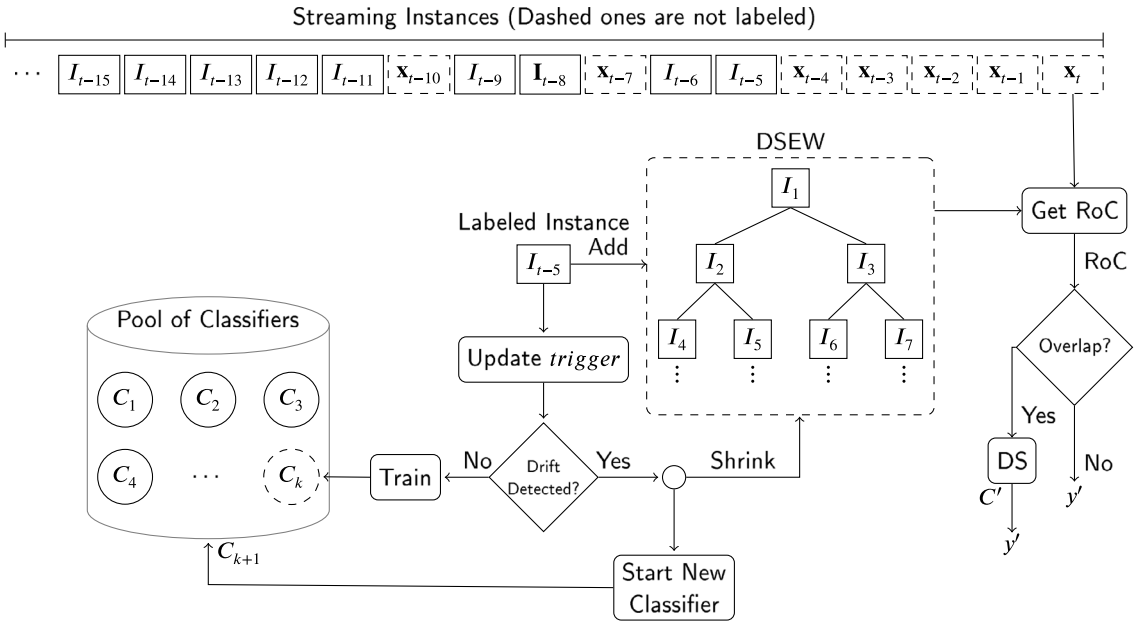


Fig. 5. IncA-DES's scheme of training and classification. The instance I_{t-5} is used for updating the *trigger*, and if a concept drift is detected, a new classifier starts to be trained, and the DSEW is shrunken. Otherwise, the last classifier C_k is trained with I_{t-5}. For classifying data point $\mathbf{x_t}$, the RoC is computed from the DSEW. If there is a disagreement between most neighbors, the DS method is used. Otherwise, the majority class in the RoC is Predicted. We represent the DSEW as a K-d Tree, but it could be any data structure depending on the neighborhood search method.

Whenever a training instance arrives, a prediction is made by the system and sent to the *trigger*. For instance, drift detectors that track the error-rate can be used, such as Drift Detection Method (DDM) [36] and Reactive Drift Detection Method (RDDM) [37]. If a concept drift is detected, the DSEW is shrunk. Thus, we avoid unnecessary adaptation that may happen in blind adaptation strategies while still being aware of concept drift. In addition, incoming training instances are added to the DSEW, ensuring it remains up-to-date, and are used for training C_k, the last classifier added to C. The rationale is that, taking into account the natural progression of data arriving over time, the classifiers are trained considering different perspectives, in different moments, favoring the generation of local experts. Additionally, old classifiers trained in past knowledge are kept as ensemble candidates, promoting both time-dependent learning and robustness.

As a counterpart of the training approach chosen for IncA-DES, by training one single classifier one by one, diversity may take longer to develop if compared to resampling strategies [11,15,32]. However, such an approach may be better if we have limited access to labeled data. The reason is that, when using a resampling training strategy, candidate classifiers may not receive enough instances for training in such cases, increasing the chances of having underfitted classifiers.

When classifying an instance, IncA-DES checks for overlap in the RoC θ, which is computed through a neighborhood search algorithm. If the rate of the majority class in θ is above a threshold ω, the class representing the neighborhood is predicted. Otherwise, the DS method is utilized. Such classification strategy helps reduce the processing time of IncA-DES, as DS methods are computationally costly. Still thinking of processing time, brute force kNN brings limitations on how big a validation set can be, as due to the continuous data stream, searching for neighbors can become computationally expensive [22]. Since data streams are concerned with processing incoming instances quickly, we propose an Online K-d tree algorithm to reduce the processing time of the RoC definition, thereby opening space for having a larger DSEW, beneficial in cases of stable concepts and *virtual concept drift*.

3.1. IncA-DES training

The pseudocode of the training procedure is in Algorithm 1. We divide the training of IncA-DES into three steps:

1. *Trigger* update (Line 1);
2. Adaptive DSEW management (Lines 2–7);
3. Time-dependent Pool Management (Lines 8–17).

In the following sections, we explain these specific topics and link them to the pseudocode.

Algorithm 1: IncA-DES Training ($Instance$)

```
   input : Labeled Instance I
1  Update trigger with I ;                    ▷ First update the drift detector
2  DSEW ← DSEW ∪ I ;                          ▷ Add I to the DSEW
3  if |DSEW| > W then
4  |  removeOldestInstance(DSEW) ;            ▷ Drop the last instance if needed
5  if concept drift was detected then
6  |  ▷ Shrink the DSEW and start a new classifier if a concept drift was detected
7  |  shrink DSEW
8  |  C_k ← a new classifier
9  |  prune(C, DSEL, C_{k-1}, D) ;            ▷ Prune a classifier based on the pruning method chosen
10 |  C ← C ∪ C_k
11 if C_k was already trained on F instances then
12 |  ▷ Start a new classifier if the last one was already trained on F instances
13 |  C_k ← a new classifier
14 |  prune(C, DSEW, C_{k-1}, D)
15 |  C ← C ∪ C_k
16 C_k ← latestClassifierAvailable(C) ;       ▷ Get C_k from C
17 train(C_k, I) ;                            ▷ Train C_k on the training instance
```

3.1.1. Trigger update and adaptive DSEW management

When a labeled instance I arrives for training, the *trigger* component is updated, as in Line 1 of the pseudocode. This is done by getting a prediction from the system and comparing the result to the true label of I. The result is sent to the *trigger*. After that, the DSEW management is performed. The new instance is added to the DSEW (Line 2), and the

oldest one is removed if it has surpassed its maximum size (Line 4). If a concept drift is detected, the DSEW is shrunk, leaving only the instances that arrived after the warning level that was given by the *trigger* (Line 7).

One may want to limit the size of the DSEW because of memory and processing time. Since the RoC definition is done through a kNN algorithm, its computational cost can increase with the growth of the search space. If there is a stable concept drift, it is preferable that as many instances as possible compose the DSEW, limited solely by the available memory. However, the processing time of the kNN algorithm becomes a concern. To mitigate this, we propose an Online K-d tree approximate neighborhood search algorithm, which is detailed in Section 3.3.

3.1.2. Time-dependent pool management

Lastly, the update of the pool C is performed by training the classifier C_k with the training instance I. C_k is always the last classifier added to the pool, which can receive a maximum of F instances for training. Once past that limit, the training of that classifier is interrupted, and the next time an instance arrives for training, a new classifier is added to C (Lines 12–16), which will be trained on the incoming instances. Moreover, the pool of IncA-DES has a limit D of how many classifiers it can have in order to prevent the infinite generation of classifiers that would lead to high memory consumption. When adding a new classifier, if C is full, the oldest one is pruned (Lines 10 and 15). We call this an age-based pruning engine, which, in contrast to dropping classifiers considering their performances, requires less processing time and does not have much difference in accuracy, as shown by Almeida et al. [16].

Furthermore, the training of C_k is interrupted not only when it has received F instances for training, but when a concept drift is detected as well (Lines 9–11). The rationale is that we guarantee that C_k is continually trained on the newest concept. The classifier's training is done after deciding whether a new classifier will be added through a concept drift or if the limit F of training was surpassed (Line 17). The F parameter brings a trade-off regarding diversity and representativeness, i.e., the size of the local region in which a classifier is trained. On the one hand, if a classifier has few instances for training, it may not have enough information to learn properly. Still, it can lead to quicker adaptation and generation of diversity. On the other hand, a bigger value for F may postpone the generation of diversity in C since we train the classifiers incrementally.

However, the optimal value for F may change according to the dataset since a more complex decision boundary may require one classifier to have more training information. At the same time, smaller groups in the feature space may get better results with a more diverse pool of candidate classifiers.

We argue that our incremental training strategy favors the generation of local experts, considering that different regions of the feature space become available in different timestamps, and we limit the region where classifiers are trained. This assumption comes from the fact that the pattern of events or people's habits is expected to change over time. As a counterpart, the diversity may take longer to be created, as one classifier is added to C at a time.

As IncA-DES trains its classifiers incrementally, it needs an incremental learner, such as the Hoeffding Tree (HT) classifier [38], which is widely used in the concept drift literature.

3.2. Classification

To classify an unlabeled test instance $\mathbf{x}$, the IncA-DES first computes the neighborhood of the test instance $\mathbf{x}$ in the DSEW to define its $\theta \subseteq DSEW$. The θ is defined as the k-nearest neighbors of $\mathbf{x}$ in the DSEW, where the distance between $\mathbf{x}$ and the instances in the DSEW can be computed using any distance function, such as the Euclidean or Canberra distances. Then, the class overlap of θ is measured. If the rate of the majority class in θ is greater than ω, the system predicts that class directly. Formally, let $s_i \in \theta$ be a subset associated with a class label $y_i \in c$, such that c is the set of possible class labels. Denote y_{maj} the majority class in θ. The decision rule is as follows:

$$\frac{|\{s_i \in \theta \mid y_i = y_{maj}\}|}{|\theta|} \geq \omega \implies return \;\; y_{maj} \tag{1}$$

If the above condition is not met, the competence of the classifiers in C is measured in the RoC, and the ensemble $C' \subseteq C$ is built based on a DS method, which can be any DS method based on RoC [20]. Then, C' classifies the instance $\mathbf{x}$ through majority voting, as presented in Fig. 5.

The most straightforward benefit of the overlap-based classification is the processing time, as the DS method is not used if most neighbors, as dictated by the ω parameter, agree with the class label. The kNN usually struggles in local regions with high class overlap, and limiting its action to regions with low or no overlap can lead to best performance. In regions with class overlap, DS usually have the best results over kNN [21].

3.3. Online K-d tree

When using a brute force kNN search to define the RoC, as the search space (i.e., number of instances) n grows, the processing time grows in $O(Kn)$, where K is the dimensionality of each sample (i.e., number of features) and n the search space. The K-d tree algorithm [39] can be used to decrease the search space, which works by building a K-dimensional binary tree data structure through recursively partitioning the feature space, and then can be used for neighborhood search.

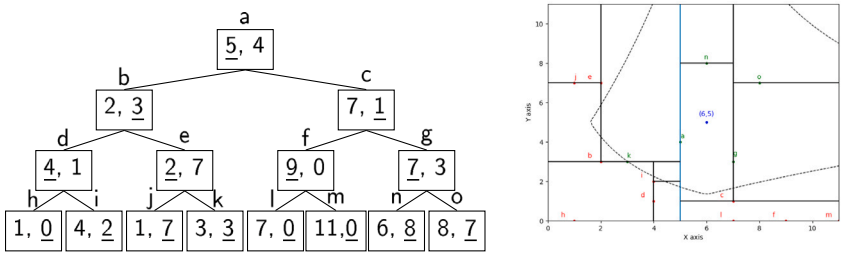
However, K-d trees were designed assuming we initially have access to the whole training set. As in data streams we have a potentially infinite flow of data, a K-d tree is required to be updated at every moment, with many insert and delete operations, which will make the data structure unbalanced. Other limitations include the need to normalize data, as the difference in the magnitude of the features may vary and harm the decrease of the search space [40]. However, data normalization is not a trivial task in data streams since the range of the features can change a lot over time, and many times using the original values of features is preferred in data streams over applying a min–max normalization, for example [41]. To address these challenges, we propose an Online K-d tree that tackles the concerns mentioned in data streams.

In order to deal with the feature magnitude issue and the need of normalization, we have used the Canberra Distance function (Eq. (2)) [42] to perform the neighborhood search. The reasons why we chose it include: (1) it normalizes the distances with the sum of the absolute values of the features; (2) it has shown to have better results than the Euclidean Distance in data streams [41].

$$d(\mathbf{x}_1, \mathbf{x}_2) = \sum_{i=1}^{K} \frac{|\mathbf{x}_1[i] - \mathbf{x}_2[i]|}{|\mathbf{x}_1[i]| + |\mathbf{x}_2[i]|} \tag{2}$$

Regarding the balancing concern in K-d trees, the initial splits of the nodes in the data structure may not be efficient when more instances arrive. Consider that one initial training data gives us the K-d tree shown in Fig. 6(a), and its divisions in two-dimensional feature space are like in Fig. 6(b). The green points are the 5-nearest neighbors of the data point $(6, 5)$ defined by the K-d tree with the Canberra distance, and the dashed contour is the domain of the Canberra Distance to the farthest neighbor.

After the arrival of more instances in a data stream, the data distribution may change, e.g., the range that features arrive is different than the training data we had initially, as exemplified in Fig. 7(a). This could happen, for instance, if we have built the K-d tree with data from the winter. Later, when the spring or summer arrives, the range of the features (e.g., the temperature) will be different.



(a) An example of a balanced K-d tree.

(b) Split of the nodes in a two-dimensional feature space. The blue vertical line refers to the split of the root node. The green data points are the 5 nearest neighbors defined by the K-d tree with the Canberra Distance.

Fig. 6. Example of a balanced K-d tree.

See that after the tree becomes unbalanced, there is not an efficient split of the nodes anymore, as shown in Fig. 7(b). If we use this structure to perform neighborhood searches, the decrease in search space may not be as efficient, depending on the depth of the structure and the data distribution. In this example, the point s is a nearest neighbor than p to the point $(6, 5)$ by the Canberra Distance, but the K-d tree algorithm pruned it from the search space.

For that reason, a common strategy found in the literature is to rebuild the K-d tree when it is two times bigger than when it was built [43,44]. This strategy, which we also adopt, ensures that the tree remains balanced even when the data distribution changes with the arrival of new instances and helps keep the neighborhood search efficient.

Next, let us define the structure of a K-d tree node. Be t a K-dimensional tree node such that $t = (I, s)$, where $I = (\mathbf{x}, y)$ is a data point, $\mathbf{x}$ is a K-dimensional feature vector, y the instance label and s is the split dimension of the node, such that $s \leq K$. The split dimension s says what is the current node's key, in which $key = \mathbf{x}[s]$. Each node t can have a left and right subtree, denoted by lt and rt, respectively. In order to maintain a K-d tree consistent, and the search of a node can be done in $O(\log n)$, as in a binary search tree, the following properties must be fulfilled:

- $\forall$ Instance $I \in lt, I[s] < I_t[s]$
- $\forall$ Instance $I \in rt, I[s] \geq I_t[s]$,

where I_t is the instance that belongs to the K-d tree node t.

3.3.1. Building a KDTree

A balanced K-d tree brings a more efficient neighborhood search by enabling better search space pruning. If a subtree holds most instances in the dataset, excluding it from the search space may speed up neighborhood search. However, discarding too many instances in an unbalanced K-d tree can reduce the effectiveness of the neighborhood search since helpful neighbors are more likely to be pruned. The pseudocode for building a balanced K-d tree is in Algorithm 2.

Firstly, we receive N instances in which the median of the split dimension ($s = 0$ in the root node) is gathered in Line 5 of the algorithm. The first instance that carries the median value will be the root node (Line 9); the instances with a smaller value than the median in the dimension s will be inserted to the left side of the root node (Line 11), and instances with higher or equal values will be to the right of the root (Line 13). This process is repeated with every set of instances recursively (Lines 15 and 16).

Algorithm 2: Build K-d Tree(Set of Instances, Split Dimension)

input : Set of Instances $instances \leftarrow \{I_1, I_2, ..., I_N\}$, $splitDimension$
output: K-d tree Node

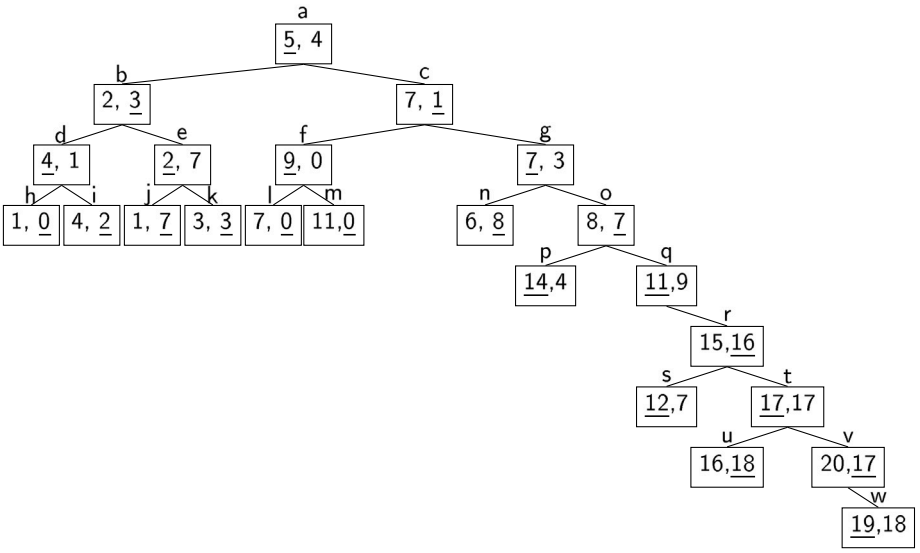
```
1  if sizeOf(instances) == 0 then
2  |  return null
3  instancesToTheLeft ← ∅
4  instancesToTheRight ← ∅
5  median ← getMedian(instances, splitDimension) ;     ▷ Get the median of the current split dimension
6  foreach I ∈ instances do
7  |  ▷ Values lower than the median go to the left subtree, and values greater or equal to the right
8  |  if I[splitDimension] == median then
9  |  |  medianInstance ← I
10 |  else if I[splitDimension] < median then
11 |  |  instancesToTheLeft ← instancesToTheLeft ∪ I
12 |  else
13 |  |  instancesToTheRight ← instancesToTheRigth ∪ I
14 node ← KDTreeNode(medianInstance, splitDimension)
15 node.left ← Build K-d Tree(instancesToTheLeft, (splitDimesion + 1) mod K)
16 node.right ← Build K-d Tree(instancesToTheRight, (splitDimension + 1) mod K)
17 return node
```

3.3.2. Insertion

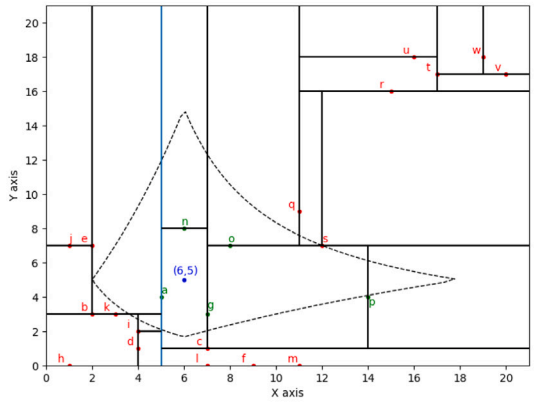
The insertion algorithm in our Online K-d tree is the same as a regular one, as described by Drozdek [45], and its pseudocode is shown in Algorithm 3. Starting with the root node, the algorithm iteratively traverses the tree by comparing the instances' values in the nodes' split dimension until it finds a leaf node (Lines 4–11). If the instance to be added has a smaller value than the current node on the split dimension, the search continues on the subtree to the left, or to the right otherwise. When the leaf node is found, if the tree is empty, it is added to the root node (Line 13). Otherwise, the value on the split dimension of its parent node is compared (Line 14). If the instance to be added is smaller than the split value of its parent node, it will be added to the left (Line 15). It is added to the right if its value is greater or equal to its parent node (Line 17).

3.3.3. Deletion

When a node is deleted, the K-d tree may become inconsistent, and the search for a node based on the key values of the split dimensions, as



(a) An example of an unbalanced K-d tree.



(b) Split of the nodes of an unbalanced K-d tree in a two-dimensional feature space. The green data points are the 5 nearest neighbors defined by the K-d tree with the Canberra Distance.

Fig. 7. Example of an unbalanced K-d tree.

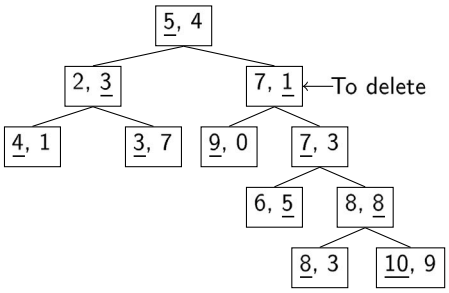
in a binary search tree, cannot be done anymore. A deletion algorithm in the K-d tree, in order to maintain its consistency, requires a lot of comparisons and arrangements of the nodes [45] or to a subtree to be entirely rebuilt. In data streams, the large flow of data brings the need to process incoming instances quickly, and a fast deletion is mandatory.

In Fig. 8, we present a step-by-step of the deletion algorithm presented by Drozdek [45]. In Fig. 8(a), there is a consistent K-d tree, and we want to delete the node (7,1). To do so, one must choose a node to replace it on the right subtree. The candidates must be split on the same dimension as (7,1), i.e., the second dimension. The candidates are the nodes (6,5) and (8,8), highlighted in Fig. 8(b). Since (6,5) has a lower value than (8,8) in the split dimension, it is chosen to replace the deleted node. The new K-d tree after the replacement is in Fig. 8(c). However, after replacing the deleted node with (6,5), one property of the subtree now is not satisfied: $\forall$ Instance $I \in rt, I[s] \geq I_t[s]$. In Fig. 8(d), we can see that the nodes (7,3) and (8,3) are in the right subtree of (6,5), but their values in the split dimension s are smaller than 5, contradicting the above property. In this data structure, the search for a node would not be done in $O(\log n)$. It would need either a depth-first or a breadth-first search.

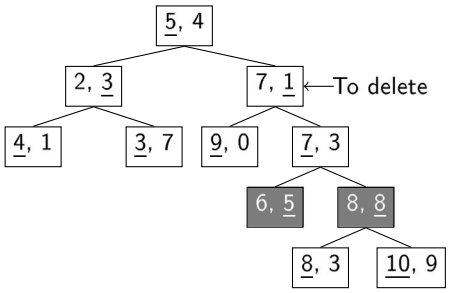
One solution would be to replace it with the node with the smallest value in the right subtree in the split dimension. However, inconsistencies could still be generated in different dimensions. Further, a more complex deletion should be performed when the node replacing the deleted one is not a leaf.

Therefore, deleting a node in a K-d tree is not a simple task. In the Online K-d tree proposed in this work, we employ a lazy deletion, which "deactivates" the deleted nodes without explicitly removing them from the data structure. The structure of the K-d tree is maintained, and the distance is not calculated on the deleted nodes. This strategy was also used by Cai et al. [46]. To do so, a flag is added to the nodes in the data structure, which tells whether the node is active, as in Fig. 9, where T stands for True, i.e., the node is active. If the node (7,1) is to be deleted, its flag will be set to False, and the resulting tree will be like in Fig. 10. Thus, we can deactivate nodes while keeping the K-d tree consistent. In this approach, the time complexity of the deletion remains solely on the search algorithm.

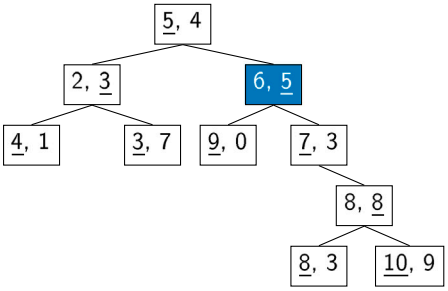
As a counterpart, this approach may lead to excessive memory usage since nodes that are not being used anymore are still maintained solely to keep a consistent structure. As we remove many instances, searching for one may become costly as the number of inactive nodes increase.



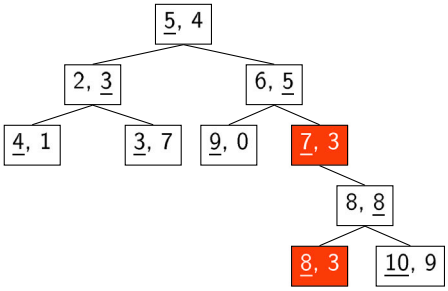
(a) An example of a 2-dimensional K-d tree. The node (7,1) is to be deleted.



(b) The nodes (6,5) and (8,8) have the same split dimension as the node to be deleted. Thus, they are candidates for replacement.



(c) As the Node (6,5) had the smallest value in the split dimension, it replaces (7,1).



(d) The nodes (7,3) and (8,3) now make the K-d tree inconsistent since their values in the split dimension are lower than (6,5). They should be on the left subtree of (6,5), not on the right.

Fig. 8. How inconsistencies can be created in a K-d tree with the deletion of a node.

Algorithm 3: K-d Tree insertion algorithm ($Instance$)

input : Instance I

$p \leftarrow KDTree.root$
$prev \leftarrow \emptyset$
$depth \leftarrow 0$
while *p is not a leaf node* **do**
  ▷ Iterate the tree until it finds a leaf node
  $prev \leftarrow p$
  if $I[depth] < p.instance[depth]$ **then**
    $p \leftarrow p.left$
  else
    $p \leftarrow p.right$
  $depth \leftarrow (depth + 1) \mod K$
if $KDTree.root == null$ **then**
  $KDTree.root \leftarrow Node(I, depth)$; ▷ Assign to the root if the tree is empty
else if $I[(depth - 1) \mod K] < p.instance[(depth - 1) \mod K]$ **then**
  $prev.left \leftarrow Node(I, depth)$; ▷ Assign to the left child if the value is smaller than the leaf node
else
  $prev.right \leftarrow Node(I, depth)$; ▷ Assign to the right child otherwise

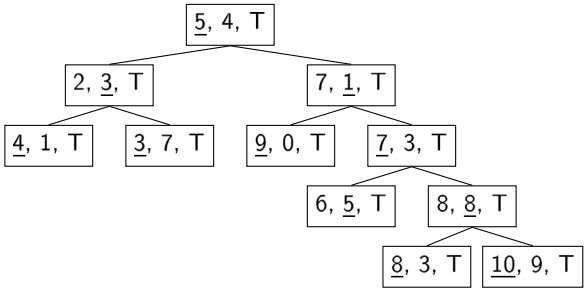


Fig. 9. 2-dimensional Tree with Flagged Nodes.

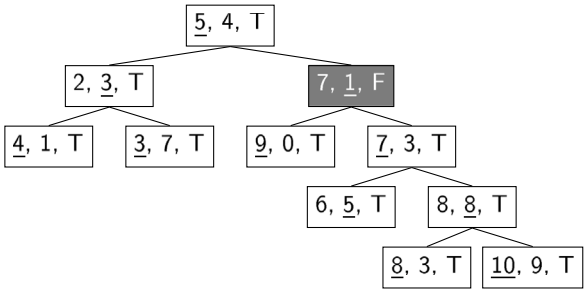


Fig. 10. 2-dimensional Tree with Flagged Nodes: A Node was Deleted.

To mitigate this issue, we set a limit of β for the proportion of inactive nodes. Once this limit is exceeded, the tree is rebuilt.

3.3.4. Neighborhood search

Once we have a K-d tree data structure, we can search for the k nearest neighbors. Since we use the Canberra Distance for the neighborhood search, a question arises: What criterion should be used to prune subtrees? The approximate neighborhood search K-d tree usually relies on the squared difference of the split dimension of the current node to do so, or to the absolute difference between features, but it uses the Euclidean Distance. With another distance function, the same split criterion may not be adequate for pruning subtrees from the neighborhood search. Instead, we use a Canberra Distance segment on

the current node's split dimension, as shown in Eq. (3).

$$seg = \frac{|I[s] - x[s]|}{|I[s]| + |x[s]|} \tag{3}$$

Thus, if the calculated seg is smaller than the current maximum distance (i.e., the largest distance among the k nearest neighbors), that subtree will be searched for neighbors. Otherwise, it is pruned from the search space. For the cases where both $I[s]$ and $\mathbf{x}[s]$ are equal to zero, which would lead to an undefined value due to a division by zero, seg will be equal to zero instead.

Algorithm 4 exposes the pseudo-code of the neighborhood search of the K-d tree. In our version of the K-d tree, we start to calculate the distances from the root node, as done by Chen et al. [47]. Line 3 checks whether the node is active, and its distance is calculated to the target test instance in Line 4. If there are fewer instances than k in the distances and neighbors list, it is added to both distances and neighborhood lists in Lines 7 and 8. If there are already k neighbors calculated, we get the maximum current distance in Line 11 and compare the calculated distance from the current node to the target instance in Line 12. If it is smaller than the current maximum neighbor, it replaces it on both distances list and neighborhood in Lines 13 and 14.

Regardless of whether a node is active, the best subtree to continue the neighborhood search is chosen by comparing the target instance's value to the current node's split dimension. If both left and right subtrees exist for the current node, the decision of which is the best one is made in Line 17. If the test instance has a split value greater or equal to the current node, the best subtree is the right one, as in Line 18. Otherwise, the best is the left subtree (Line 21). If only one subtree exists, be the right or the left, they are set as the best (Lines 23–26). If we have found a leaf node, the algorithm returns the neighbors found so far (Line 28).

Then the decision of whether to search or not the subtree is done in Line 30, which is where the segment of the canberra distance in the split dimension s is calculated by using Eq. (3), and is compared to the current maximum distance. If the algorithm says that the best subtree should be search for neighbors, the search continues recursively in Line 31. This is done on the subtree other than the best one also (Lines 33–35). In the end, the k neighbors are returned in Line 36.

Algorithm 4: Neighborhood Search ($Node, \mathbf{x}, distances, neighbors, k$)

input : KDTreeNode $Node$, Target Instance $\mathbf{x}$, List $distances$, List $currentNeighbors$, Number of Neighbors k
output: Neighbors, Distances

1 $I \leftarrow Node.instance$
2 $s \leftarrow Node.splitDimension$
3 **if** $Node\ is\ Active$ **then**
4 $distanceToNode \leftarrow distanceFunction(I, \mathbf{x})$ ▷ Calculate distance if the node is active
5 **if** *Size of* $distances$ *is smaller than* k *neighbors* **then**
6 ▷ Add distance and instance to the lists if its size is smaller than k
7 $distances.add(distanceToNode)$
8 $neighbors.add(Node.instance)$
9 **else**
10 ▷ Otherwise, tests whether the current maximum distance is greater than the distance to the current node
11 $maximum, maxIndex \leftarrow getMaximum(distances)$
12 **if** $distanceToNode \leq maximum$ **then**
13 $distances[maxIndex] \leftarrow distanceToNode$
14 $neighbors[maxIndex] \leftarrow Node.instance$
15 **if** $Node.left \neq null$ *and* $Node.right \neq null$ **then**
16 ▷ Checks what is the best subtree to continue the search
17 **if** $\mathbf{x}[s] \geq I[s]$ **then**
18 $best = Node.right$
19 $other = Node.left$
20 **else**
21 $best = Node.left$
22 $other = Node.right$
23 **else if** $Node.left \neq null$ **then**
24 $best = Node.left$
25 **else if** $Node.right \neq null$ **then**
26 $best = Node.right$
27 **else**
28 return $neighbors$
29 $maximum \leftarrow getMaxDist(distances)$
30 **if** $Should\ Search\ SubTree(Node, \mathbf{x}, s, maximum)$ **then**
31 $neighbors, distances \leftarrow NeighborhoodSearch(best, target, distances, neighbors, \mathrm{k})$ ▷ Continue neighborhood search on the best subtree
32 $maximum \leftarrow getMaxDist(distances)$
33 **if** $other \neq \emptyset$ **then**
34 **if** $Should\ Search\ SubTree(Node, \mathbf{x}, s, maximum)$ **then**
35 $neighbors, distances \leftarrow NeighborhoodSearch(other, target, distances, neighbors, \mathrm{k})$ ▷ Search the other subtree too if needed
36 return $neighbors$

4. Related works

We may find various approaches for dealing with concept drift in the literature, such as blind adaptation [9,10,16,48,49], use of drift detectors [37,50–52], ensembles [9,10,30], and Dynamic Selection (DS) [15,16].

We focus on describing methods based on Triggers (Section 4.1) and ensemble (Section 4.2) since the proposed IncA-DES combines these two approaches. For an analysis that includes other strategies to deal with concept drifts, refer to [2,53].

4.1. Concept drift detection

Concept drift detectors, in the majority, monitor the classification error. An increase in the classification error may be evidence of concept drift, and different detectors employ different tests in order to trigger a concept drift. The Page–Hinkley Test (PH) continuously performs statistical tests on the error rate. When it rises over a predefined threshold, it is told that a change has happened [54]. The DDM [36], one of the most well-known drift detectors, also monitors the error rate of classification and considers different thresholds for warning and drift levels. The warning level is where the concept drift started, which is given when the error rate rises above a certain level. If the error rate keeps growing and gets to the drift level, a concept drift has happened. The Early Drift Detection Method (EDDM) [55] extends DDM by taking into account the distance between two misclassifications through the usage of distance functions. RDDM [37] also extends the DDM by triggering a concept drift when the warning level stays for too long.

Statistical Test of Equal Proportions Detector (STEPD) [56] comes from the principle that accuracy remains similar as new instances come if the concept is stable, and a decrease in accuracy may indicate a concept drift. The difference in accuracy is checked through a statistical test, in which when the p-value is smaller than a threshold, a concept drift is detected.

Hoeffding Drift Detection Method (HDDM) [57] applies either the Hoeffding's Inequality [58] (HDDMA) or the McDiarmid's Inequality [59] (HDDMW) for detecting changes in the moving average of

streaming data. Accurate Concept Drift Detection Method (ACDDM) [60] also uses Hoeffding's Inequality, monitoring the prequential error. EWMA for Concept Drift Detection (ECDD) tracks the classification error by using Exponentially Weighted Moving Average [61]. Adaptive Windowing (ADWIN) maintains a sliding window W and continuously compares statistics of two different data portions inside W. When the statistics of the subwindows differ from each other, a concept drift is triggered [62].

Fast Hoeffding Drift Detection Method (FHDDM) [63] uses the assumption that, in stable concepts, the accuracy may either remain unchanged or increase. A concept drift is triggered if the accuracy decreases above a certain level. Hoeffding's Inequality compares the maximum accuracy gathered to the current one. Stacking Fast Hoeffding Drift Detection Method (FHDDMS) [64] maintains a short and a long window by considering that a short window is best for abrupt concept drift and the larger one for gradual concept drift. FHDDMS$_{add}$, from the same work, replaces the binary calculation by the summation of the recent points.

SeqDrift1 performs hypothesis tests on two different batches of the data [65]. If the hypothesis is rejected, the batches are combined, and the hypothesis test is repeated when a new batch is available. Authors have used the Bernstein bound [66] on the hypothesis test. SeqDrift2 extends SeqDrift1 by applying reservoir sampling, which the authors argue to result in a tighter threshold for drift detection [67].

McDiarmid Drift Detection Method (MDDM) uses McDiarmid's Inequality on statistical tests applied on a sliding window in which earlier instances have a higher weight than the oldest. There are three different versions of the MDDM, where each one uses a different weight function MDDMA uses the arithmetic weighting, MDDMG the geometric weighting function, and MDDME an exponential function.

Recent works have also been focusing on unsupervised drift detection. Uncertainty Drift Detection (UDD) uses the uncertainty of predictions of neural networks as the error rate [68]. Label Dependency Drift Detector (LD3) considers the changing correlation of the predicted labels overtime [51]. Student–Teacher approach for Unsupervised Drift Detection (STUDD) uses two classification models: a student and a teacher [52]. The teacher is trained on the true labels of the dataset, while the student is trained using the teacher's predictions on the same dataset. The concept drift is tracked by taking into account the difference between the predictions of both models, and when this difference between the two models rises above a threshold, a concept drift is triggered. Thus, there is no need for the true label of the data, as the error rate is calculated through the difference of previously trained models (some access to labels is still needed initially).

4.2. Ensemble learning for concept drift

Ensemble learning algorithms can be divided into two categories: static and dynamic. Static ensembles are those that the ensemble of classifiers is defined during the training phase. All of the classifiers are trained by following a training strategy, such as bagging [69]. Dynamic ensembles, differently from static, define the ensemble during the test phase. This is done by gathering the most promising classifiers according to the characteristics of the test instance. In this section, we explore methods of both types found in the literature.

4.2.1. Static ensemble

Many of the ensemble methods for data streams (static or dynamic) use online bagging, or OzaBag [32] to train the classifiers. Online bagging simulates the classic bagging algorithm used in static ML by using the Poisson distribution with $\lambda = 1$, which the authors show to be an approximation of the sampling with replacement procedure from the static bagging algorithm [69]. Oza [32] also presented Online Boosting, in which if an instance is misclassified by a model, its weight is increased when shown to other classifiers to be trained. Otherwise, it is decreased. This process focus training on instances that are hard to be classified, as on the classic boosting algorithm [70].

Later on, authors have used the online bagging algorithm with different values for the λ parameter. Leveraging Bagging [71] have extended the Online Bagging by employing $\lambda = 6$, and by the addition of the ADWIN drift detector to adapt to concept drift, aiming to replace outdated classifiers. Most of the ensemble methods for concept drift that use online bagging seems to prefer to use $\lambda = 6$. By taking into account the Poisson distribution, if $\lambda = 6$, we have that $Poisson(X \geq 1) = 99.75\%$, i.e., there is a high chance that all of the classifiers will be trained on the incoming instances at least once. This may lead to low-diversity ensemble, given that most of the classifiers are trained on all of the instances. This λ value was also adopted by Adaptive Random Forest (ARF), which trains HT classifiers through online bagging, and each classifier carries its own drift detector [11]. In ARF, drifted classifiers are replaced by new ones trained in recent data.

Minku and Yao [72], coming from the assumption that high-diversity ensemble is better after concept drift, and low-diversity is preferable under stable concepts [28], proposed Diversity for Dealing with Drifts (DDD), which maintains two ensembles trained with different λ values. The low-diversity ensemble is trained by using $\lambda = 1$. High diversity is induced by employing lower values for λ, which in DDD is chosen by empirically testing different values for different datasets. When concept drift is detected, the high diversity ensemble is used to perform the predictions. When the concept is stable, the low diversity ensemble is used.

Streaming Ensemble Algorithm (SEA) [73] trains classifiers on the incoming batches of data. The ensemble is updated only if the new classifier trained on the latest labeled batch is able to improve the ensemble's performance. Accuracy Weighted Ensemble (AWE) [74] weights the classifiers based on their performance to the most recent batches of data, i.e., classifiers that better classify the last arrived batch are given higher weights when voting. Accuracy Updated Ensemble (AUE) [75] includes incremental learners and updates the classifiers' weights based on the current data distribution. The inclusion of incremental learners, different from AWE, makes the ensemble members of AUE able to be incrementally updated. Thus, it is able to update the classifiers on newest labeled instances.

Online Accuracy Updated Ensemble (OAUE) [76], as the name suggest, is an online version of the AUE algorithm. OAUE updates the ensemble one instance at a time, characterizing an online processing of data (see Section 2.2). The weights of the models are immediately updated on the latest instance, ensuring a more effective real-time adaptation to concept drift.

Comprehensive Active Learning Framework for Multiclass Imbalanced Data Streams with Concept Drift (CALMID) utilizes a hybrid labeling strategy, combining randomness, uncertainty, and a selective sampling strategy to choose samples to be requested for labels [30]. In CALMID, arrived labeled samples are weighted according to the difficulty and imbalance rate of its class. The ensemble is then trained on the weighted sample by using online bagging. CALMID also keeps two different sliding windows: (1) label sliding window, to be aware of the class distributions; (2) sample sliding window, which is used to store the most recent instances of each class. Concept drift is addressed through the ADWIN drift detector. When ADWIN detects a concept drift, a new classifier is trained on the sample sliding window and is added to the ensemble. Then the less accurate classifier is pruned from the ensemble.

Ensemble Optimization for Concept Drift (EOCD) [77] dynamically updates the weight of classifiers in an ensemble based on their performance over time. Initially, classifiers are trained on the initial data, with the most accurate ones set as active and the rest as inactive. Classifiers whose weights fall below a threshold are replaced by inactive classifiers, and a new inactive classifier is trained on the most recent instances. An optimization technique is employed to select new active

classifiers and their weights, while the remaining classifiers are moved to an inactive set and trained offline. Concept drift is detected using RDDM [37], and adaptation involves choosing new ensemble members and training new classifiers.

Hybrid Ensemble approach to concept drift-tolerate transfer learning (HE-CDTL) aims at the transfer learning domain, using an adaptive weighted correlation alignment to improve the transfer of knowledge between domains [10]. Like SEA, classifiers are trained on the incoming batches of data, and are combined through a class-wise ensemble strategy. Selective Ensemble-based Online Adaptive Neural Network (SEOA) [9] uses two different types of neural networks: deep and shallow. The rationale is that shallow neural networks are able to learn a new concept more quickly if compared to deep neural networks. The choice of which one to use is done based on the error-rate. A high error-rate indicates a changing distribution, thus the shallow neural network is used. Otherwise, the deep neural network is used for the predictions. In other words, the deep neural network is used in stable concepts, and shallow neural networks under concept drift.

Streaming data classification algorithm based on hierarchical concept drift and online ensemble (SCHCDOE) maintains the model untouched under stable concepts. When data distribution is changing (warning level is given), SCHCDOE updates its model by using random subspaces [78]. Under concept drift, abnormal data are discarded, and the training is performed by combining ensemble learning and incremental learning. Cost-sensitive Continuous Ensemble Kernel Learning method (CCEKL) aims at the class imbalance issue [79]. The authors present a misclassification cost to adapt to changing class distribution. The adaptation to concept drift is done through a continuous kernel learning method.

4.2.2. Dynamic ensemble

DS approaches for dealing with data streams with concept drift usually train classifiers on the incoming instances by adopting a batch-based processing or some strategy of online processing. The idea is to maintain an updated pool of classifiers C and to keep the earliest instances as the DSEL. Attribute-Oriented Dynamic Classifier Selection (AO-DCS) constructs the subsets in which the competence of the classifiers is measured through the values of the features instead of using the most commonly used neighborhood-based approach [80]. Incoming instances are divided into data chunks, in which each one is used for training a base classifier and also constructing an evaluation set Z, which can be considered as the DSEL. Numerical values are discretized, and instances that share the same values in the same features belong to the same subset. The competence of each classifier is measured on the subset of the test instance, and the single classifier with the best average accuracy in the subset is chosen for classification.

The Dynse Framework [16] trains classifiers in batches of data arrived in time and uses a sliding window as a DSEL in order to always maintain recent instances to measure the competence of the classifiers. Double Dynamic Classifier Selection (DDCS) [13] applies resampling to update the classifiers in C, while still training classifiers in new batches of data. Dynamic Ensemble Diversification (DynEd) [15] uses diversity-based ranking in order to select the classifiers, and when a drift is triggered by an ADWIN detector, a new classifier is trained on the last instances from a sliding window, and is added to the pool C. Classifiers in C are updated with resampling, in which the λ value changes depending on the pool diversity and fluctuations in the accuracy. A K-Means algorithm is also applied on C to divide the classifiers into groups before performing the DES itself, in which the most accurate classifiers on a sliding window are selected.

Zyblewski et al. [81] uses stratified bagging and preprocessing strategies to deal with the imbalanced class problem in data streams. Incoming data chunks are subjected to sampling with replacement, preserving the number of instances for both minority and majority classes. A classifier is trained on the resulting data. The arrived data chunks are the DSEL that will be used to perform the DES. Dynamic Ensemble Selection for Imbalanced Data Streams with Concept Drift (DES-ICD) also aims at the imbalanced class issue by using oversampling strategies [17]. They compare the class ratio from the most recent batch of data to the previous one and generate new samples from the minority class by considering the degree of change in the distribution. Dynamic Ensemble Selection based on Window over Imbalanced Drift Data Stream (DESW-ID) tries to overcome the imbalanced data problem by adopting resampling strategies by training more classifiers with the minority class. The DS strategies use a reverse search algorithm to pick the best number of classifiers for an ensemble. The error rate sorts the classifiers, and the authors have used ADWIN as a concept drift detector in order to shrink the adaptive window in which the classifiers are ranked [14]. Adaptive Bagging-based Dynamic Ensemble Selection (AB-DES) is a chunk-based algorithm that employs a dual adaptive sampling technique for training the classifiers [18]. AB-DES stores data chunks obtained from an undersampling method and applies oversampling to minority classes. In addition, newer data chunks have a higher weight than the oldest ones, and a sliding window-based drift detector is used to adapt to concept drift.

Assis et al. [22] argue on the unfeasibility of using neighborhood-based DS methods in data streams and propose Mass-Based Short Term Selection (MSTS), in which the RoC are the ten last instances in the data stream. Thus, there is no need to do a neighborhood search.

Adaptive Regularized Ensemble (ARE) [82] has induced diversity by training HTs in different random subspaces and used only incorrectly classified instances for the training. Classifier selection is performed by choosing the classifiers that are able to correctly classify the last arrived instances, as done in MSTS [22]. Adaptive Random Tree Ensemble (ARTE) [83] has used the same structure of ARF for training, maintaining an ensemble with the members carrying each a drift detector. In ARTE, each ensemble member is trained on a different random feature subspace. Additionally, the cut points for splitting the trees are chosen randomly. The ensemble members are also chosen based on the performance to the most recent instances.

In Table 1, we present a concept matrix of the described DS methods for concept drift. Many of the cited methods rely on the brute force kNN to define the RoC. The proposed framework uses the Online K-d tree to define the RoC aiming to mitigate the limitation of the validation set and adopt a different training strategy than other methods.

5. Experiments

In this section, we define the datasets used in the experiments, present the experimental protocol, and compare the proposed framework to the state-of-the-art methods. Additionally, we do an Ablation Study comparing different components of IncA-DES and a Case Study where we assess the scalability of the Online K-d tree when the search space increases.

5.1. Datasets

In this work, we divide datasets into real-world datasets and synthetic datasets. The real-world datasets offer more realistic scenarios in which we do not know about the presence or nature of concept drift. Synthetic datasets contain induced concept drift, making them applicable to evaluate the behavior of a ML model to known changing distributions.

Table 2 exposes the datasets used in the experiments and their characteristics. They were gathered from the USP DS Repository [85], the MOA Repository [86], and the UCI [87]. The synthetic datasets were generated using the Massive Online Analysis (MOA) framework [86]. To the SEA, Sine STAGGER and Agrawal datasets, 1,000,000 instances were generated, and the concept has changed abruptly every 10,000 instances, characterizing a recurrent concept drift. On the Hyperplane and LED datasets, we have a gradual concept drift starting at the 50,000th instance, and the new concept stabilizes at the 50,500th

Table 1
Concept matrix of DS methods for concept drift.

Method	Type	Drift detector	RoC definition	Training
AO-DCS [84]	Batch	–	Attribute-Oriented	Streaming Batches
Dynse [16]	Batch	–	brute force kNN	Streaming Batches
DDCS [13]	Online	–	brute force kNN	Online Bagging
[81]	Batch	–	brute force kNN	Streaming Batches
DES-ICD [17]	Online	ADWIN	brute force kNN	Streaming Batches
DynEd [15]	Online	ADWIN	–	Online Bagging
DESW-ID [14]	Online	ADWIN	Sliding Window	Streaming Batches
MSTS [22]	Online	–	Sliding Window	Online Boosting
AB-DES [18]	Batch	Sliding Window-based	brute force kNN	Adaptive Bagging
ARE [82]	Online	ADWIN	Sliding Window	Online Bagging
ARTE [83]	Online	ADWIN	Sliding Window	Online Bagging
IncA-DES	Online	Any	Online K-d tree	Incremental

Table 2
Datasets used in the experiments. A: Abrupt, G: Gradual, I: Incremental, R: Recurrent, V: Virtual.

Dataset	Source	Type	# Instances	# Features	# Classes
Real-world					
Electricity	[88]	Unknown	45,312	8	2
Nursery	[89]	Unknown	12,960	8	5
Ozone	[90]	Unknown	2536	72	2
Gas Sensor	[91]	Unknown	13,910	128	6
Adult	[92]	Unknown	45,222	14	2
Rialto	[93]	Unknown	82,250	27	10
NOAA	[5]	Unknown	18,159	8	2
Keystroke	[94]	Unknown	1600	10	4
Covertype	[95]	Unknown	581,012	54	7
Yeast	[96]	Unknown	1484	8	10
Asfault	[97]	Unknown	8066	62	5
Insects-AB	[85]	A	52,848	33	6
Insects-GB	[85]	G	24,150	33	6
Insects-IB	[85]	I	57,018	33	6
Letter	[98]	V	20,000	16	26
Digits	[99]	V	5620	64	10
Pen Digits	[99]	V	10,992	16	10
Dry Bean	[100]	V	13,611	16	7
Rice	[101]	V	3810	7	2
Synthetic					
SEA	[73]	A, R	1,000,000	3	2
STAGGER	[102]	A, R	1,000,000	3	2
Sine	[36]	A, R	1,000,000	2	2
Agrawal	[103]	A	1,000,000	9	2
Hyperplane	[104]	G	100,000	10	2
LED	[69]	G	100,000	24	10
RandomRBF	[105]	I	100,000	10	2

instance. On the RandomRBF dataset, there is an incremental concept drift throughout the whole stream. For the Hyperplane, LED, and RandomRBF datasets, 100,000 instances are generated. Other details about these datasets are the same as the default in the MOA framework.

Furthermore, on the Letters, Digits, Pen Digits, Dry Bean, and Rice datasets, a *virtual concept drift* was induced by dividing the data stream into feature spaces, as done by Almeida et al. [16]. To do so, in each step, a single random instance is chosen from the dataset, and the 199 nearest neighbors to it according to the Euclidean Distance are sent to the data stream. Thus, we use a chunk of 200 instances, as it is our training size. This process is repeated until the dataset is empty. This way, we simulate an environment where different regions of the feature space become available with time. Fig. 11 shows the steps of this process. All of the datasets used in the experiments of this work are available in our GitHub repository.[2]

[2] https://github.com/eduardovlb/IncA-DES

5.2. Experimental protocol

The experiments in this work were designed to compare different methods and approaches in different scenarios, i.e., different dimensionality, sample size, and label availability. To do so, various real-world and synthetic datasets were used and two different training policies were adopted. The first one is the test-then-train, where every instance is used for testing and then for training [1]. Such a policy considers that all of the labels are available for the dataset without delay.

The second training strategy introduces two limitations in the training policy to make testing closer to real-world environments: (1) delayed arrival of labels and (2) partial availability of the labels. The first limitation is straightforward: a delay is applied to when a label becomes available after it was used to test the model's prediction performance. Since the delay of labels in the real world varies according to the problem, it is difficult to define how long a label takes to arrive. For instance, in weather forecasts, information about whether it has rained a day is available the next day. As a counterpart, other datasets require a longer delay for the labels and often require human action. Gomes

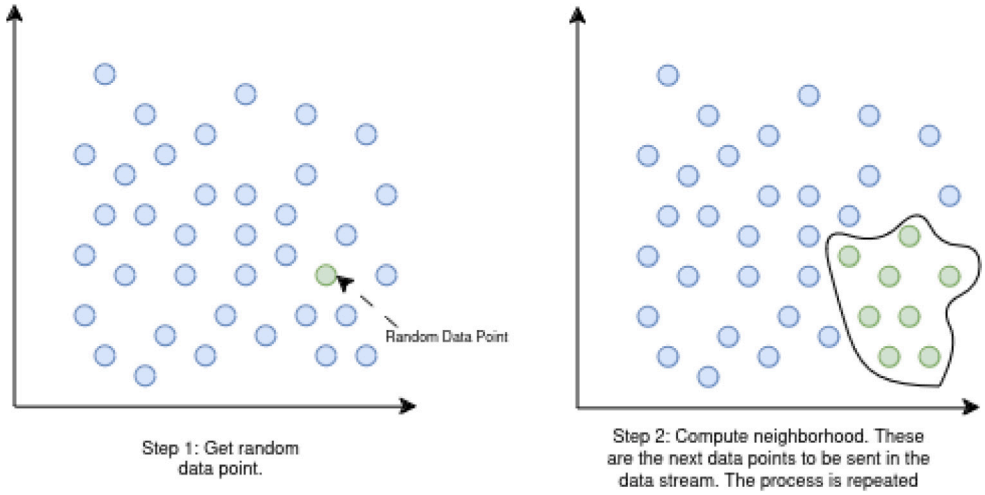


Fig. 11. Steps for simulating *Virtual Concept Drift*.

et al. [11] employs a delay of 1000 instances, which we use in this work, in addition to two other levels: equal to our training size (200) and 1/4 of the training size (50).

On the second limitation, we considered that, for every two instances in the data stream, only one is labeled, which we call partial labeling. These two limitations were introduced together with the aim of better simulating an environment in the real world. The chosen evaluation metrics were the average accuracy, the prequential accuracy on a sliding window of 1000 instances [1], and the average number of Instances per second (I/s).

The baseline state-of-the-art methods used for comparison include three DS methods for concept drift and four ensemble methods for concept drift, which are listed below:

- ARE: A DS method that trains classifiers only on the instances that the ensemble is not able to classify correctly. Each classifier is trained in a different random region of the subspace to induce diversity, and the classifiers to compose the ensemble are chosen according to their performance in the last arrived labeled instances [82].
- Dynse: A batch-based DS method that trains classifiers whenever a batch of labeled data arrives in the timestamp [16].
- DynED: An online DS method for concept drift that divides the classifiers into different groups through a K-means algorithm and chooses the best ones to compose an ensemble based on diversity. The training is done through an online bagging algorithm that updates the λ parameter with time [15].
- ARF: A well-known ensemble-based method for concept drift where each inner classifier carries a concept drift detector. Classifiers are trained through online bagging, and whenever a single classifier is considered outdated by the drift detector, it is replaced [11].
- OzaBag: This is the original online bagging, which adapts the batch-based bagging [69] to online learning [32].
- LevBag: An extension to OzaBag, which increases the λ parameter and includes output detection codes [71].
- OAUE: An online variation of AUE, which trains its classifiers on incoming instances, weights them based on the prediction error and frequently replaces non-accurate classifiers with new ones [76].

Before the comparison, a hyperparameter tuning for IncA-DES was performed, where the impact of each hyperparameter (*trigger*, D, F, k and ω) is measured individually. More details can be found in Appendix A. The default configuration after hyperparameter tuning is exposed in Table 3. Datasets used for the hyperparameter tuning (Electricity, NOAA, Nursery, and Digits) were excluded from the comparison with the state-of-the-art methods. Details about the state-of-the-art hyperparameters are in Appendix B.

Table 3
Set of Hyperparameters of IncA-DES. D is the pool size, F is the maximum number of instances a classifier receives for training, k is the number of neighbors, ω the rate of the majority class in the RoC, and β is the proportion of inactive nodes in the Online K-d tree.

Hyperparameter	Value
W	Limited by Memory
$Trigger$	RDDM
D	75
F	200
Pruning Engine	Age-based
DS method	KNORAE
k	5
ω	0.8
Distance Function	Canberra Distance
β	0.3
Base Classifier	Hoeffding Tree

Since the DSEW has its size limited solely by the memory, none of the datasets tested needed a deletion operation in IncA-DES. Therefore, the β parameter defined for the K-d tree did not influence IncA-DES. Despite that, after some empirical study, we recommend setting $\beta = 0.3$, i.e., if 30% of the nodes are inactive, the tree is rebuilt. This should give a good balance between binary search time and an effective neighborhood definition.

Later, we assess how the components settled for IncA-DES (K-d tree and Overlap-based classification) influence accuracy performance and processing time. In these experiments, seven real-world datasets with different dimensionalities were used to cover different scenarios equally. In the end, we have used three synthetic datasets to evaluate how the Online K-d tree differs from the brute force kNN in different search spaces. Since with the synthetic datasets we can control the environment and generate as many instances as needed, different sizes for the DSEW, i.e., the search space n, were considered to compare how the K-d tree influences the accuracy, processing time and I/s. Only one concept of the datasets was generated, and each search space was used to label 100,000 instances.

The experiments were performed using Java 17 running the MOA framework [86], except for the DynED [15], which was built in Python 3.8 on the scikit-multiflow library [106]. All of the results reported are an average of 10 runs.

Table 4
Accuracies of IncA-DES and methods of the state of the art. The W-T-L row counts the number of wins, ties, and losses of the methods compared to the proposed framework.

Dataset	IncADES	ARE	Dynse	DynED	ARF	OzaBag	LevBag	OAUE
Ozone	93.84	**94.50**	93.62	93.25	94.32	93.96	93.86	93.96
Gas Sensor	**96.76**	91.01	91.19	94.92	91.12	55.45	82.51	71.93
Adult	83.26	83.05	82.18	83.49	84.14	**84.43**	84.25	84.35
Rialto	**82.43**	81.83	79.71	74.02	72.82	31.01	60.40	52.90
Keystroke	**96.49**	94.98	84.81	88.90	94.20	82.91	88.81	74.86
Insects-AB	**75.73**	74.50	72.32	74.60	75.17	57.61	69.13	64.64
Insects-GB	**79.38**	77.69	75.61	76.16	78.03	61.86	71.99	59.69
Insects-IB	61.70	**65.93**	60.60	64.11	65.70	54.33	61.35	56.92
Yeast	53.27	55.91	49.07	**56.40**	51.17	49.67	49.77	44.24
Asfault	88.25	87.81	85.66	88.38	**88.67**	71.49	87.06	77.53
Covertype	93.00	**95.47**	92.82	94.69	94.88	83.66	91.77	90.03
Pen Digits	**95.55**	94.49	93.23	90.02	92.61	86.51	89.53	86.97
Dry Bean	89.66	**90.62**	89.86	90.12	88.47	89.50	89.23	88.00
Rice	91.43	92.02	90.13	91.79	**92.42**	91.61	91.76	88.57
Letters	**85.34**	50.82	82.66	65.30	58.70	61.94	64.33	63.81
SineRec	93.98	96.76	89.80	**99.14**	96.98	57.06	93.36	90.62
SeaRec	87.05	87.98	85.87	**89.05**	87.89	84.32	87.16	86.77
StaggerRec	**99.91**	99.71	99.70	99.73	99.79	72.36	98.02	97.87
Agrawal	83.60	80.99	79.00	**94.07**	80.36	61.97	81.17	85.03
Hyperplane	87.48	88.55	**89.83**	84.76	86.91	87.35	87.74	88.43
LED	69.52	73.72	70.00	73.22	**73.83**	**73.83**	73.78	73.72
RandomRBF	93.47	**94.23**	86.30	93.05	**94.23**	89.04	93.40	91.58
Average	**85.50**	84.10	82.91	84.51	83.75	71.90	81.38	77.84
Av. Rank	**3.23**	**3.23**	5.32	3.45	3.27	6.41	4.86	6.05
W-T-L	–	12-0-10	19-0-3	11-0-11	12-0-10	18-0-4	16-0-6	17-0-5

5.3. Comparing with state of the art

5.3.1. Experiments with test-then-train

Table 4 shows the average accuracy of the tested methods and the number of wins (W), ties (T), and losses (L) of IncA-DES to the respective methods. Notice that IncA-DES got the best average accuracy, followed by DynED and ARE, and it also had more wins than losses compared to all of the state-of-the-art methods, except DynED, to which there was a tie. IncA-DES and ARE were tied in the first place in the average rank.

In Fig. 12, we see a plot of accuracy versus I/s of the state-of-the-art methods, where we can see that IncA-DES, when compared to ARE, DynED and ARF, was not only the most accurate one but also the one that was able to process more instances in one second, although it uses neighborhood search for defining the RoC. More specifically, it processed 1.25 times more instances than ARE in one second, 34.57 times more instances than DynED, and 3.98 times more instances than ARF. IncA-DES also relies on a simpler training policy, where classifiers are trained incrementally one by one. It looks a lot like the training approach used in Dynse [16], but due to the higher DSEW and the need to compute the neighborhood in the training phase as well (to update the drift detector), it was not faster than Dynse.

Although other methods like OzaBag and OAUE were the fastest, they were the least accurate — OzaBag had a difference of 13.60 percentage points to IncA-DES and OAUE of 7.67 percentage points. LevBag and Dynse got good accuracy with more I/s, but still not as accurate as IncA-DES, ARE, or DynED. For the interested reader, Table 17, found in Appendix C, shows the average I/s for each method.

Now let us evaluate the windowed prequential accuracy of the methods, shown in Fig. 13. We excluded the OAUE and OzaBag methods from the graphs for visualization purposes since they are the least accurate. Considering the Ozone dataset (Fig. 13a), we see that ARE outperformed the other methods in most of the timestamps. On the Gas Sensor dataset, in Fig. 13b, IncA-DES had the highest accuracy on most of the timestamps and did not present drops in accuracy at many moments where other methods did. Although it did not have the best accuracy on the Adult dataset, in Fig. 13c, we can notice that the prequential accuracy of the methods was quite the same throughout the timestamp.

On the Rialto dataset (Fig. 13d), IncA-DES, at the moments where all of the methods present drops in accuracy, was the one that struggled less and had higher accuracy on most of the moments. ARE was competitive on this one. On the Keystroke dataset in Fig. 13e, IncA-DES is seen as the most accurate at every moment. On the Insects benchmark, IncA-DES was competitive on the datasets with abrupt and gradual concept drifts (Figs. 13f and 13g). However, on the Insects-IB dataset (Fig. 13h), which carries an incremental concept drift, IncA-DES was not able to be competitive with ARE, ARF or DynED. ARE presented the best prequential accuracy in every moment in the Yeast dataset (Fig. 13i), and many methods kept a similar accuracy in the timestamp of the Asfault dataset (Fig. 13j). IncA-DES has presented some drops in accuracy on the Covertype dataset in Fig. 13k, probably due to the presence of categorical features, that neighborhood search cannot handle well, as also suggests Dynse's performance.

Getting into the datasets with an induced *virtual concept drift*, IncA-DES was the best performing on the Pen Digits and Letters datasets, in Figs. 13l and 13o, presenting the best accuracy on all of the timestamps and a significative dominance on the Letters. On the Dry Bean and Rice datasets (Fig. 13m and 1n), DynED appears as the most accurate in most of the timestamps.

On the synthetic datasets, we see IncA-DES struggling less on the Sine dataset (Fig. 13p) after the first concept drift. Still, on the subsequent ones, it seems that the decision boundaries are not adequately learned, presenting a smaller accuracy on the 3rd and 4th concepts. This is perceived in the remaining of the stream. On the STAGGERRec dataset, in Fig. 13s, it was the one who struggled the less with the drifting concepts and did not present the drops in accuracy that the other methods did after concept drift. We see dominance from DynED on the Agrawal dataset (Fig. 13t), and a greater accuracy from Dynse and ARE on the Hyperplane dataset (Fig. 13).

On the LED dataset (Fig. 13u, the neighborhood-based methods (IncA-DES and Dynse) seem to struggle, also likely due to the presence of categorical features, as in the Covertype dataset. On the RandomRBF dataset (Fig. 13v), we see that IncA-DES, ARE and ARF got a similar accuracy at many moments, but yet ARF and ARE were the best performing.

Finally, we have performed a Wilcoxon Signed-Rank Test, where we can directly compare two different methods. The p-values rounded to two decimals are in Table 5. Notice that IncA-DES only did not present

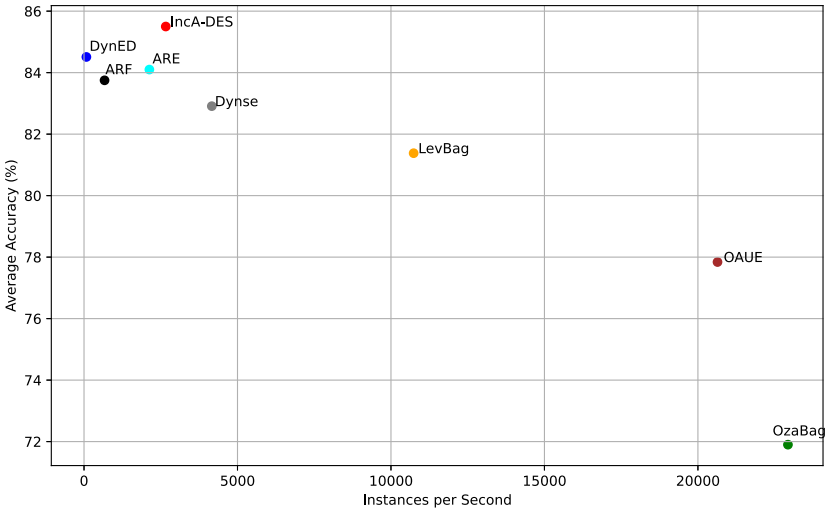


Fig. 12. Plot of average accuracy versus average instances per second of the state-of-the-art methods on all tested datasets.

Table 5
P-values of the Wilcoxon Signed-Rank Test. P-values smaller than 0.05 are set in bold, and the (±) sign indicates whether the column-wise method has surpassed the row-wise method.

	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE	IncA-DES
ARE	–	**0.00 (-)**	0.73	0.50	**0.00 (-)**	**0.00 (-)**	**0.00 (-)**	0.98
Dynse		–	0.06	**0.04(+)**	**0.00 (-)**	0.81	**0.02 (-)**	**0.00 (+)**
DynED			–	0.94	**0.00 (-)**	**0.00 (-)**	**0.00 (-)**	0.68
ARF				–	**0.00 (-)**	**0.00 (-)**	**0.00 (-)**	0.39
OzaBag					–	**0.00 (+)**	**0.00 (+)**	**0.00 (+)**
LevBag						–	**0.00 (-)**	**0.00 (+)**
OAUE							–	**0.00 (+)**
IncA-DES								–

a statistically significant difference to ARE, DynED and ARF, but still with a higher average accuracy.

In conclusion, we can say that IncA-DES was the method with the best average accuracy, the fastest between the most accurate methods, and obtained a good predictive accuracy in most of the scenarios tested, while presenting more wins than losses compared to every tested method, even with a simpler training approach. However, it seems to struggle on some datasets, such as Ozone, Insects-IB, Covertype, Sine, and LED, where lower accuracy was perceived in many moments of the timestamps. On the Sine dataset, the struggle is perceived mainly on the 3rd and 4th concepts. It also seems to struggle under slow changing or long stable concepts, as suggests the performance in the Insects-IB dataset. The adopted training strategy may also harm performance in some scenarios, although it can lead to better performance in others. If the data is sent in a meaningful and natural order over time, the classifiers can cover each a different local region, favoring the usage of local-based DS. Otherwise, the performance can be compromised.

5.3.2. Experiments with delayed and partial labels

Now, let us analyze the results with a limited label availability. In Fig. 14, we show the accuracies of the test-then-train policy and the variations of each method with each delay. IncA-DES still got the highest average accuracy in all of the scenarios. Notice that out of the most accurate methods, it was the one that had the smallest variation in accuracy as the availability of the labels became limited. Although OzaBag presented the smaller variation, its accuracy performance was always poor compared to the other methods. Thus, the proposed framework presented resilience with less access to training data when compared to other methods. We hypothesize that the adopted training strategy of IncA-DES helps in scenarios with limited labeled data since the training is focused on one classifier.

Now, we present the Wilcoxon signed-rank tests with different delays in Table 6. See that for delays equal to 50 and 200, IncA-DES only did not have a statistically significant difference to ARE, DynED and ARF. When the delay was equal to 1000, ARE has presented statistically significant difference to five methods, while IncA-DES to two. ARE seems not to lose much performance of most of the datasets when label is limited, while IncA-DES presented some substantial loss in some cases (see Tables 18, 19, and 20 in Appendix D), making the difference to other methods not be significant according to the Wilcoxon test.

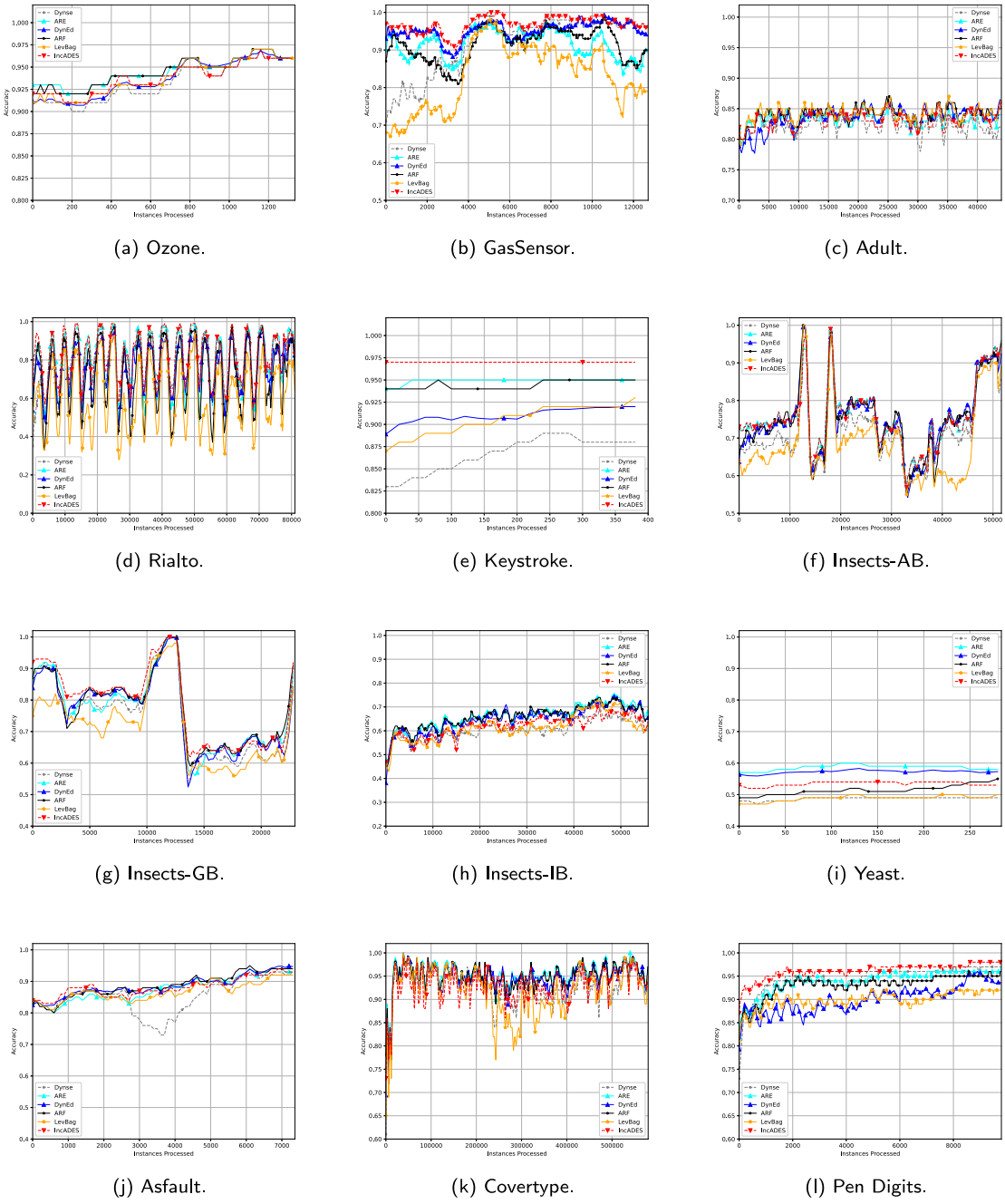
In conclusion, limiting the label availability has interfered with the results of all the tested state-of-the-art methods, and some struggled more than others. DynED had the highest variation in accuracy with a high delay, with a difference of 17.55 percentage points from the delay of 1000 instances to the test-then-train policy.

Considering IncA-DES, ARE, ARF, and DynED, the most accurate methods in the test-then-train policy, IncA-DES had the smallest difference in accuracy on every delay, thus being the most resilient under limited training data considering average accuracy. However, ARE was the one that got a significant difference from most of the methods with the longest label delay (1000).

5.4. Ablation study: Evaluating components of IncA-DES

Now that we have compared the proposed framework to the state-of-the-art methods let us assess how the proposed Online K-d tree algorithm and the overlap-based classification influence both accuracy performance and processing time of IncA-DES. For these experiments, the datasets were selected to cover different dimensionality scenarios: Electricity, NOAA, Adult, Gas Sensor, Covertype, Insects-AB, and Rialto. In these tests, the fact that the K-d tree is rebuilt sometimes influences the processing time. The ω parameter has been set to 1 in order to use the classification filter with no overlap only.

In Table 7, we first present the average accuracy, processing time, and instances per second of a baseline version of IncA-DES, which uses the brute force kNN for defining the RoC and does not consider an



(a) Ozone. (b) GasSensor. (c) Adult.

(d) Rialto. (e) Keystroke. (f) Insects-AB.

(g) Insects-GB. (h) Insects-IB. (i) Yeast.

(j) Asfault. (k) Covertype. (l) Pen Digits.

Fig. 13. Prequential accuracies of IncA-DES and other state-of-the-art methods on test-then-train.

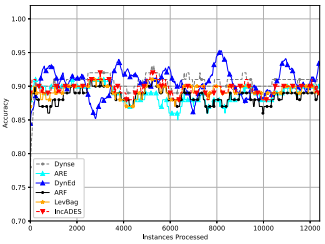
overlap-based classification. The subsequent columns expose the results when different components are added: the overlap-based classification, the Online K-d tree, and, finally, both components combined.

We can see that the framework got more benefits with the overlap-based classification, which improved accuracy and processing time in all tested datasets. Regarding the K-d tree, we can see a drop in average accuracy but an improvement in processing time on some datasets. Its benefits seem more problem-specific, a constraint already known for K-d trees. The processing time may be harmed if the algorithm cannot effectively split the subtrees from the search space. This can happen due to, for instance, a data distribution that does not favor the partition through the Canberra Distance.

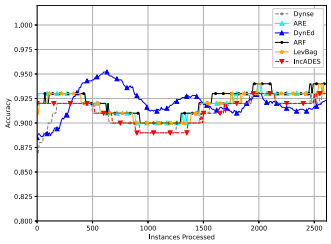
In conclusion, we can say that the Online K-d tree sped up the RoC definition on some datasets. However, due to the non-optimal RoC given by the K-d tree, the accuracy performance decreased but at a negligible level. The overlap-based classification improved both accuracy and processing time. Therefore, we recommend that authors think of using this type of classification filter when dealing with DS methods based on RoC and choose carefully when to use the Online K-d tree algorithm over the brute force kNN. If one is more concerned with the accuracy performance, it may prefer to rely on the brute force kNN. However, in some cases, the Online K-d tree algorithm can decrease the processing time substantially without much impact on accuracy.

5.5. Case study: Varying the search space — size of DSEW

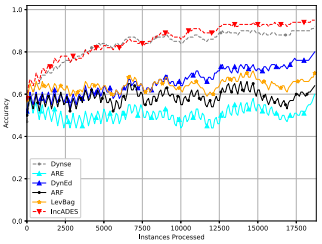
In this section, we extend the experiments for the Online K-d tree algorithm by performing tests varying the size of the DSEW in the IncA-DES framework. The Sine, SEA, and Agrawal datasets were considered for these experiments and only instances of the first concept of these datasets were generated. For each value of n, IncA-DES



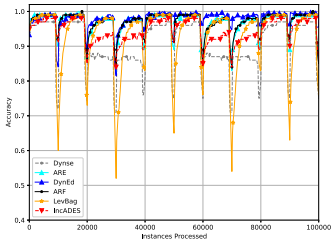
(m) DryBean.



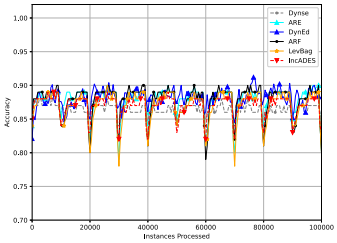
(n) Rice.



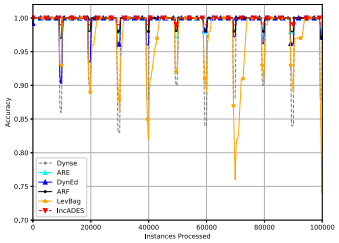
(o) Letter.



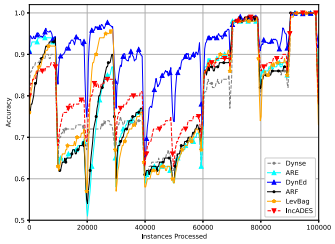
(p) Sine-Rec.



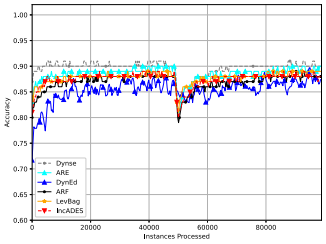
(q) SEA-Rec.



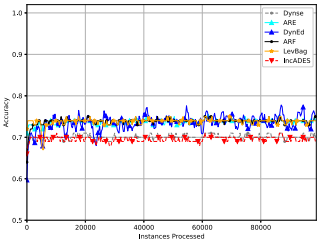
(r) STAGGER-Rec.



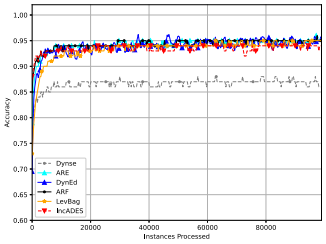
(s) Agrawal.



(t) Hyperplane.



(u) LED.



(v) RandomRBF.

Fig. 13. (*continued*).

classified 100,000 instances. The idea is to compare the difference in performance and processing time as the search space n increases. In these experiments, the K-d tree is not rebuilt. Thus, the analysis is focused on classification. In Table 8, we show the accuracies, processing time, and how many instances each approach can classify per second.

On average, the Online K-d tree had the smallest processing time and the highest number of classified instances per second. Regarding performance, except for when $n = 1000$, the brute force kNN consistently achieved the highest accuracy, but the difference to the K-d tree is negligible. Although IncA-DES did not achieve a smaller processing time on the Agrawal dataset, the difference got smaller as the search space increased. The difference between the Sine and SEA datasets is quite significant. See that IncA-DES managed to label more than 24 times the number of instances that the brute force kNN could label in one second when $n = 50{,}000$.

In Fig. 15, we can see the variation in processing time as the search space increases. As seen previously, the Online K-d tree does not suffer on the Sine and SEA datasets, while on the Agrawal dataset, its processing time increases as n grows. Possible explanations are that the data distribution or characteristics may not allow effective search space partitioning. If the number of distance calculations is close to the brute force kNN, the overhead required by the K-d tree (e.g., construction and traversal) leads to increased processing time.

In conclusion, the proposed Online K-d tree algorithm presented quicker neighborhood definition compared to the brute force kNN, mostly when the search space increases. However, its benefits are not guaranteed in every scenario, a constraint prior known from the K-d tree. Although the neighborhood definition can be sped up, the cost of maintaining a K-d tree cannot be ignored. One must choose carefully when to use the K-d tree and when to remain with the brute force kNN

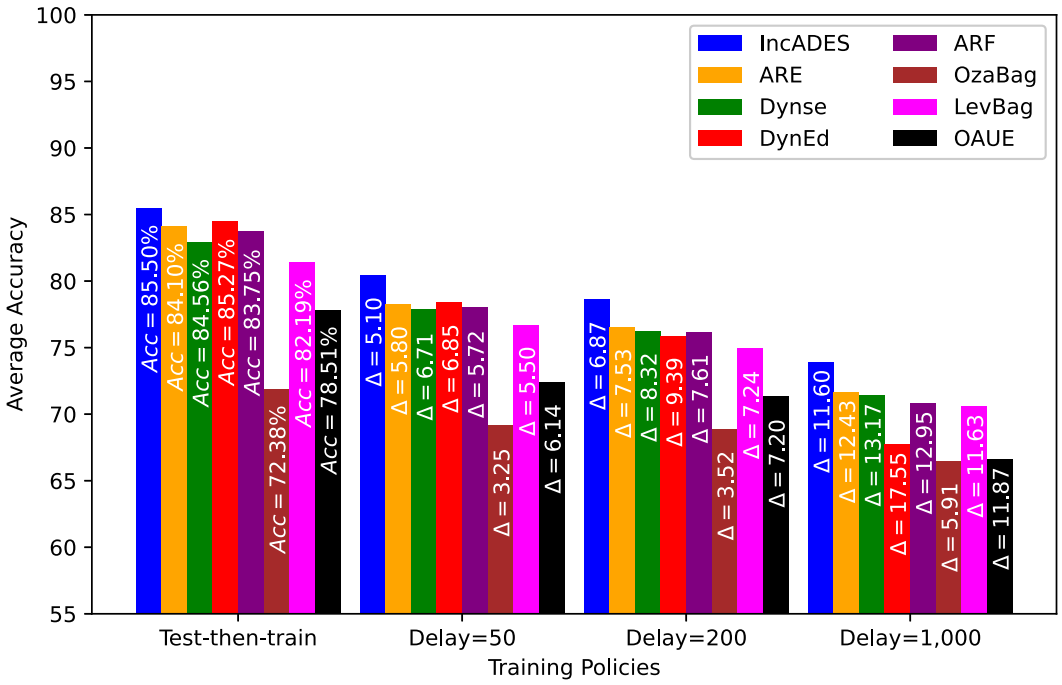


Fig. 14. Variations of accuracy for each value of delay of labels. The average accuracy of the test-then-train policy is shown, and the variation Δ compared to the test-then-train is shown for the other training policies. Notice that IncA-DES always had the highest average accuracy for every training policy.

Table 6
P-values of the Wilcoxon Signed-Rank Test with delayed and partial labeling. P-values minor than 0.05 are set in bold, and the (±) sign indicates whether the column-wise method has surpassed the row-wise method.

				Delay = 50				
	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE	IncADES
ARE	–	**0.04 (-)**	0.34	0.35	**0.00 (-)**	**0.01 (-)**	**0.00 (-)**	0.24
Dynse		–	0.39	0.34	**0.00 (-)**	0.31	**0.00 (-)**	**0.00 (+)**
DynEd			–	0.96	**0.01 (-)**	0.22	**0.00 (-)**	0.27
ARF				–	0.01	0.08	**0.01 (-)**	0.28
OzaBag					–	**0.00 (+)**	0.23	**0.00 (+)**
LevBag						–	**0.00 (-)**	**0.00 (+)**
OAUE							–	**0.00 (+)**
IncADES								–
				Delay = 200				
	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE	IncADES
ARE	–	**0.02 (-)**	0.19	0.36	**0.01 (-)**	**0.00 (-)**	**0.01 (-)**	0.50
Dynse		–	0.57	0.32	**0.00 (-)**	0.35	**0.00 (-)**	**0.01 (+)**
DynEd			–	0.57	0.05	0.39	**0.01 (-)**	0.31
ARF				–	**0.02 (-)**	0.1	**0.03 (-)**	0.25
OzaBag					–	**0.03 (+)**	0.28	**0.00 (+)**
LevBag						–	**0.01 (-)**	**0.00 (+)**
OAUE							–	**0.00 (+)**
IncADES								–
				Delay = 1000				
	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE	IncADES
ARE	–	**0.04 (-)**	**0.04 (-)**	0.24	**0.03 (-)**	**0.04 (-)**	**0.01 (-)**	0.76
Dynse		–	0.32	0.64	0.06	0.35	**0.01 (-)**	0.11
DynEd			–	0.07	0.73	0.26	0.4	0.09
ARF				–	0.16	0.34	**0.05 (-)**	0.24
OzaBag					–	0.15	0.64	**0.02 (+)**
LevBag						–	**0.02 (-)**	0.05
OAUE							–	**0.01 (+)**
IncADES								–

6. Conclusion

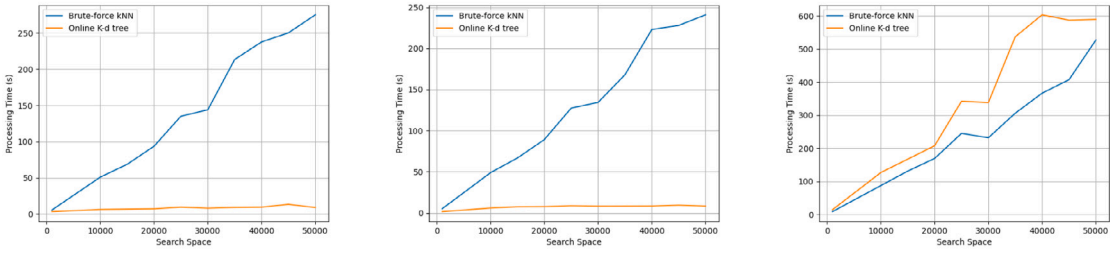
In this work, we have presented a novel framework to deal with data streams with concept drift called IncA-DES, which combines concept drift detectors with dynamic selection of classifiers. An adaptive mechanism is used in order to keep more instances from a stable concept in the DSEW, and drift detectors assist in the maintenance of information when a concept drift is triggered by shrinking the validation set. Additionally, we introduce two components to promote effectiveness in neighborhood-based DS applications: 1) an Online K-d tree Algorithm for approximate neighborhood search, which employs lazy deletion to avoid inconsistencies generated with the removal of nodes, and (2) an overlap-based classification filter that leverages the kNN algorithm when the neighbors of the test instance mostly agree with the class.

Table 7
Average accuracy, processing time and instances per second of IncA-DES with a baseline configuration (brute force kNN without overlap-based classification) comparing with the addition of the components separately (K-d Tree and overlap-based classification).

Dataset	kNN no overlap			kNN overlap			K-d tree no overlap			K-d tree overlap		
	Accuracy	Time (s)	I/s	Accuracy	Time (s)	I/s	Accuracy	Time (s)	I/s	Accuracy	Time (s)	I/s
Electricity	88.85	18.37	2456	**88.89**	8.76	5150	**88.88**	17.49	2579	88.65	**8.32**	**5422**
NOAA	75.11	8.79	2043	**75.32**	6.47	2776	75.30	8.40	2138	75.18	**6.43**	**2793**
Adult	83.11	318.08	142	**83.31**	241.05	187	83.06	341.89	132	83.25	**192.09**	**234**
Gas Sensor	94.44	83.72	164	96.68	**15.03**	**912**	94.40	94.09	146	**96.70**	24.56	558
Rialto	80.62	693.36	118	**82.35**	473.53	173	79.90	386.87	212	81.66	**193.57**	**424**
Insects-AB	74.46	279.3	188	75.07	**171.01**	**308**	74.43	505.67	104	**75.12**	287.64	183
Covertype	93.59	2719.04	214	**93.68**	1512.46	384	93.43	2233.85	260	93.44	**1046.72**	**555**
Average	84.31	588.67	760	**85.04**	346.90	1413	84.20	512.61	796	84.86	**251.33**	**1453**

Table 8
Accuracies, processing time, and instances per second of brute force kNN and our proposed Online K-d tree algorithm. n stands for the DSEW's size. On the I/s column, values between parenthesis refer to how many times more instances the Online K-d tree could label compared to the brute force kNN in one second. Each search space was used to label 100,000 instances with IncA-DES.

Dataset	kNN			K-d tree		
	Accuracy	Time (s)	I/s	Accuracy	Time (s)	I/s
n=1000						
Sine	95.94	5.51	18,149	**95.98**	**3.31**	**30,211** (1.66x)
SEA	87.27	5.05	19,802	**87.36**	**1.52**	**65,789** (3.32x)
Agrawal	**85.56**	**8.75**	**11,429**	85.49	14.59	6854 (0.60x)
Average	89.59	**6.44**	16,460	**89.61**	6.47	**34,285** (2.08x)
n=10,000						
Sine	**98.17**	50.94	1963	98.15	**6.30**	**15,873** (8.09x)
SEA	**88.42**	48.93	2044	88.40	**5.87**	**17,036** (8.33x)
Agrawal	**91.17**	**87.48**	**1143**	90.95	126.81	789 (0.69x)
Average	**92.59**	62.45	1717	92.50	**46.33**	**11,233** (6.54x)
n=25,000						
Sine	**98.64**	134.98	741	98.58	**9.47**	**10,560** (14.25x)
SEA	**88.62**	127.36	785	88.58	**8.56**	**11,682** (14.88x)
Agrawal	**92.01**	**244.94**	**408**	91.71	341.89	292 (0.72x)
Average	**93.09**	169.09	645	92.96	**119.97**	**7511** (11.64x)
n=50,000						
Sine	**98.77**	275.08	363	98.75	**8.95**	**11,173** (30.78x)
SEA	**88.82**	240.84	415	88.79	**8.20**	**12,195** (29.39x)
Agrawal	**92.23**	**526.91**	**190**	91.90	589.17	170 (0.89x)
Average	**93.27**	347.61	323	93.15	**202.11**	**7846** (24.29x)



(a) Sine. (b) SEA. (c) Agrawal.

Fig. 15. Variation of the processing time from IncA-DES with the variation of the search space for the brute force kNN and the Online K-d tree.

Experimental analysis has shown that the proposed framework had best overall accuracy than the 7 tested methods from the state of the art on 22 datasets in four different scenarios of label availability and delay of instances in data stream, and presented statistically significant difference to four methods on most scenarios. ARE, a method from the state of the art, also presented good resilience when label delay was introduced, but IncA-DES still was the most accurate.

In addition, the Online K-d tree algorithm improved the processing time of the proposed framework in some scenarios with a negligible loss in accuracy, and the overlap-based classification led to improvements in both accuracy and processing time. However, an open issue still involves the curse of dimensionality, which is a common issue for both brute force kNN and K-d tree, in which distance functions lose their effectiveness under high-dimensional problems. To address this issue, feature selection techniques can be employed, or DES approaches that do not rely on the neighborhood of the test instance can also be considered.

For future works, we intend to evaluate different training strategies, pursuing the best one in terms of diversity and generation of local experts for DS in drifting scenarios. Furthermore, aiming to address the natural trend of labels being delayed and partial in the real world,

Table 9
Initial set of Hyperparameters before the tuning. F is the maximum training size of each classifier, D is the maximum size of the pool of classifiers, k the neighborhood size of the RoC for the KNORA-Eliminate, and ω is the rate of the majority class in the neighborhood.

Parameter	Value
F	200
D	25
k	5
ω	1

we will also explore options of unsupervised, self-supervised, and semi-supervised learning, which may bring best results in such scenarios. Since the neighborhood search still seems able to evolve in streaming scenarios, we also intend to test different ways to do it in data streams, and to better address the curse of dimensionality. Additionally, imbalanced data streams is also a constant problem under high-scale data streams. Futurely, we will explore alternatives to overcome this issue, such as oversampling or undersampling techniques [14,17].

CRediT authorship contribution statement

Eduardo V.L. Barboza: Writing – original draft, Visualization, Methodology. **Paulo R. Lisboa de Almeida:** Writing – review & editing, Writing – original draft, Supervision. **Alceu de Souza Britto:** Writing – review & editing, Supervision. **Robert Sabourin:** Writing – review & editing, Visualization, Supervision, Funding acquisition. **Rafael M.O. Cruz:** Writing – review & editing, Visualization, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Eduardo Victor Lima Barboza reports financial support was provided by Natural Sciences and Engineering Research Council of Canada. Eduardo Victor Lima Barboza reports financial support was provided by Coordination of Higher Education Personnel Improvement. Alceu de Souza Britto Jr. reports financial support was provided by Brazilian National Council for the State Funding Agencies. Eduardo Victor Lima Barboza reports equipment, drugs, or supplies was provided by Digital Research Alliance of Canada. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was enabled in part by NSRC-Canada (Natural Sciences and Engineering Research Council of Canada), CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior), CNPq (Conselho Nacional de Pesquisa) and by support provided by the Digital Research Alliance of Canada (alliancecan.ca).

Appendix A. Hyperparameter analysis

This section presents the experiments performed to tune the hyperparameters of IncA-DES. We start with an initial set of hyperparameters, shown in Table 9, which will be tuned individually throughout this section. The primary metric that we used for evaluating them is accuracy. The drift detector is not exposed because it is the first hyperparameter to be tuned. The Hoeffding Tree [38] was defined as the base classifier.

We assume that, as we use a K-d tree to perform neighborhood search, we can let the DSEW grow more and worry less about the processing time for defining the RoC. Thus, we did not set a limit for the maximum number of instances in the validation set. The only constraint, in this case, would be the memory. Other hyperparameters, such as the DS method and the pruning engine, were fixed. We used the KNORA-Eliminate [107], which is widely used in the literature and usually presents good results. As the pruning engine, we used the age-based, since Almeida et al. [16] showed that it is less costly and leads to results similar to an accuracy-based one. The overlap-based classification and the Online K-d tree for defining the RoC, components presented in this work, are also set as default.

A.1. Concept drift detectors

In this Section, we have tested 8 different drift detectors on IncA-DES. The drift detectors used in these tests were introduced in Section 4. Table 10 shows the average accuracy on all datasets. We see that the RDDM drift detector achieved the highest average accuracy, even though it did not have individual victories over any of the tested datasets. Nevertheless, as the results indicates that it is the most consistent, it is chosen as the default drift detector for IncA-DES.

A.2. Maximum training size of classifiers — F

In this Section, we compare the accuracies of different values for F, which dictates the maximum number of instances a classifier can receive for training. When C_k has received F instances for training, its training is interrupted, and another online classifier is added to C and starts to be trained. Minor values for F will help to add more classifiers in C earlier in the stream, but may not give enough information. On the other hand, higher values for F may lead to a slower building of a diverse pool of classifiers. The results are exposed in Table 11. As we can see, $F = 200$ got the best average accuracy and is set as the default.

A.3. Size of pool of classifiers — D

The D parameter also brings trade-offs regarding drift adaptation since we use an age-based pruning engine. Higher values of D maintain older classifiers for longer, which still would be candidates for classifying incoming instances. Remember that this would not happen if we were using an accuracy-based pruning engine, but the need to check the performance of classifiers would require more processing time. Maintaining old classifiers for longer may be beneficial when

Table 10
Average accuracies of IncA-DES with different concept drift detectors.

Dataset	DDM		EDDM		RDDM		STEPD		ADWIN		HDDMA		HDDMW		FHDDM	
	Acc.	# Det.	Acc.	# Det.	Acc.	# Det.	Acc.	# Det.	Acc.	# Det.	Accuracy	# Det.	Accuracy	# Det.	Accuracy	# Det.
Nursery	93.84	25.1	**93.99**	19.6	93.82	21.9	93.54	18.8	92.72	5	92.55	4.6	91.74	2	91.81	3
Electricity	85.78	57.1	**89.15**	133.5	88.17	82.6	88.79	106	85.85	13.5	86.68	32.7	87.44	35.3	86.78	20.5
NOAA	75.76	6.6	74.59	76.6	75.17	20.1	75.03	34.3	**76.16**	1	75.98	3	74.92	25.3	75.19	23
Digits	93.18	0	91.54	3.4	93.14	0	89.71	8.3	93.17	0	93.10	0	**93.21**	0	93.14	0
Average	86.95	22.2	87.15	58.3	**87.58**	31.2	86.84	41.9	87.09	4.9	87.16	10.1	86.72	15.7	86.68	11.6

Table 11
Accuracies of IncA-DES with different values for F.

Dataset	50	100	200	500	1000
Nursery	92.55	92.99	93.87	**94.81**	94.59
Electricity	87.12	87.54	**88.40**	87.56	87.82
NOAA	**75.50**	75.38	75.27	75.07	74.91
Digits	91.86	92.74	**93.15**	92.83	92.70
Average	86.76	87.19	**87.63**	87.57	87.51

Table 12
Accuracies of IncA-DES with different values for D.

Dataset	25	50	75	100
Nursery	**93.87**	92.86	93.81	**93.87**
Electricity	88.40	88.51	88.86	**88.90**
NOAA	**75.27**	75.26	75.17	**75.27**
Digits	93.15	93.14	93.15	**93.18**
Average	87.67	87.69	87.75	**87.81**

Table 13
Accuracies of IncA-DES with different values for k.

Dataset	5	7	9	11
Nursery	**93.81**	93.74	93.58	93.60
Electricity	**88.86**	88.67	88.12	88.10
NOAA	75.17	75.63	75.87	**76.87**
Digits	**93.15**	92.68	92.22	91.90
Average	**87.75**	87.68	87.45	87.47

Table 14
Accuracies of IncA-DES with different values for ω.

Dataset	1	0.8
Nursery	93.81	**93.94**
Electricity	**88.86**	88.25
NOAA	75.17	**75.57**
Digits	93.15	**93.98**
Average	87.75	**87.94**

Table 15
Dynse hyperparameters.

Hyperparameter	Value
D	100
M	2/32
B	200
Pruning Engine	Age-based
DS Method	KNORAE
k	5

we have recurrent concept drifts, in which their information would be useful again when the concept returns. The D parameter aligned with the F parameter dictates both the diversity and the longevity of a concept in C (remember, if we use an age-based pruning engine). The results of different D values are in Table 12. We see that $D = 100$ got the higher accuracy. However, since the difference to $D = 75$ was not significant, we will choose to use $D = 75$ to provide quicker adaptation and processing time for the framework.

A.4. Neighborhood size — k

Now, we test the impact of the neighborhood size k of the RoC for the KNORA-Eliminate in the framework. This hyperparameter dictates the size of the local regions where the classifiers' competence is measured, which plays a crucial part in neighborhood-based DS frameworks [13,16].

Table 13 shows the average accuracy for each value, where we see that $k = 5$ was the most accurate one. Thus, it is set as the default. However, we must mention that the optimal size for the region of interest can vary according to the data distribution of the dataset and on how the local regions are distributed.

A.5. Rate of overlap — ω

In this section, we analyze the ω parameter, which dictates when the DS method will be used to classify an instance, or if the majority class of the RoC will be used to label the test instance. The results are in Table 14, where we see that $\omega = 0.8$ has achieved the highest accuracy, i.e., if 4 out of the 5 neighbors in the RoC belong to the same class, that class is used to label the test instance. Thus, allowing a bit of overlap in the RoC in the classification filter seems to bring benefits, as we can better leverage the kNN algorithm. That said, $\omega = 0.8$ is set as default in IncA-DES.

Appendix B. State-of-the-art hyperparameters

We have separated the hyperparameters from Dynse to other state-of-the-art methods for some reasons. Firstly, as the authors had different configurations for different data sizes, we have performed our own tuning for Dynse for some hyperparameters on the same datasets from IncA-DES. The tuned hyperparameters were D, M, and B. Other values were left as the default value, as well as the usage of two different values of M for different types of concept drift (*real* or *virtual*) done by the authors. Secondly, Dynse has more hyperparameters than the other state-of-the-art methods, which most of them do not share. Dynse's hyperparameters are shown in Table 15.

The hyperparameters of the other state-of-the-art methods were maintained as the default configuration and are in Table 16.

Table 16
Hyperparameters of other state-of-the-art methods.

Method	Hyperparameters		
DynEd	D 1000	W 1000	Drift Detector ADWIN
ARF	D 100	Drift Detector ADWIN	
OzaBag	D 10		
LevBag	D 10		
OAUE	D 10	W 500	

Appendix C. Table of processing time of the state-of-the-art methods

Table 17 shows the processing time in seconds of all tested methods in this work.

Appendix D. Tables of accuracies for delayed labels

Tables 18–20 show the average accuracies for IncA-DES and all of the tested state-of-the-art with delays equal to 50, 200, and 1000, respectively.

Table 17
Average instances per second of IncA-DES and methods of the state of the art.

Dataset	IncA-DES	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE
Ozone	1743	550	3708	42	142	**10,618**	2539	11,124
Gas Sensor	667	236	806	13	86	**1785**	981	1186
Adult	202	900	3875	174	301	**24,737**	6660	10,303
Rialto	574	761	1169	47	235	**7000**	3719	4127
Keystroke	3500	1333	11,667	196	625	**17,500**	7778	46,667
Insects-AB	227	784	1148	26	244	**6937**	3512	4091
Insects-GB	264	822	1702	37	266	**6709**	3507	4246
Insects-IB	43	668	1140	38	236	**6584**	3300	3785
Yeast	3669	1284	11,673	102	600	18,343	9171	**64,200**
Asfault	630	347	1829	29	160	**3894**	2103	3250
Covertype	854	635	1236	21	231	**10,645**	4313	5347
Pen Digits	342	1296	872	45	262	**8919**	4072	5996
Dry Bean	2521	2447	736	67	471	**11,764**	5683	7577
Rice	10,939	3505	3471	232	1068	**36,100**	14,440	32,818
Letters	134	857	507	9	121	3867	1735	**2215**
SineRec	9301	4971	10,671	73	1591	**50,251**	27,352	37,398
SeaRec	15,029	3270	6773	129	958	**67,568**	31,827	49,776
StaggerRec	6987	16,396	10,436	57	5230	93,545	69,881	**98,814**
Agrawal	747	1123	7048	48	336	**24,522**	9659	15,840
Hyperplane	71	1353	4572	123	369	**38,610**	11,520	18,797
LED	52	1820	1872	58	573	**17,452**	1355	9302.33
RandomRBF	88	1486	4671	130	541	37,175	11,136	17,182
Average	2663	2129	4163	77	669	**22 933**	10 738	20 638

Table 18
Accuracies of IncA-DES and methods of the state of the art with delayed and partial labels (Delay = 50).

Dataset	IncA-DES	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE
Ozone	91.79	93.75	**93.96**	93.00	93.84	**93.96**	93.16	**93.96**
Gas Sensor	**71.53**	64.86	69.08	66.15	61.32	58.12	61.90	55.57
Adult	83.05	82.94	82.54	82.91	83.45	84.21	84.08	**84.25**
Rialto	**40.94**	40.76	39.70	39.14	34.23	28.41	30.60	30.59
Keystroke	**94.71**	92.24	78.99	80.10	90.79	71.43	83.07	56.21
Insects-AB	**73.85**	71.97	70.08	72.37	71.43	54.86	66.43	62.43
Insects-GB	**76.23**	74.58	71.74	73.36	74.23	56.92	68.43	60.15
Insects-IB	60.59	64.51	59.63	62.39	**64.53**	49.54	59.52	55.40
Yeast	**48.86**	43.99	45.74	38.60	34.35	45.95	44.55	34.81
Asfault	85.39	83.40	81.63	84.25	**85.51**	70.60	78.19	69.99
Covertype	88.82	**92.42**	91.90	91.68	91.76	80.30	88.57	85.06
Pen Digits	**92.72**	90.22	89.05	83.06	89.40	81.55	85.73	81.32
Dry Bean	88.17	**88.88**	87.84	88.33	87.99	88.78	88.38	85.76
Rice	90.71	91.39	89.32	90.21	**91.82**	90.52	91.53	83.22
Letters	74.76	38.29	**74.81**	55.24	49.20	59.15	58.98	59.31
SineRec	92.44	91.49	88.22	**98.20**	95.02	55.95	90.77	84.99
SEARec	86.34	86.12	86.29	**88.64**	86.84	84.26	86.34	85.97
StaggerRec	**99.58**	99.07	97.23	99.20	99.29	71.40	98.42	95.51
Agrawal	81.95	78.59	77.75	**93.08**	77.31	61.32	77.71	81.38
Hyperplane	86.72	87.32	**89.20**	83.75	86.05	87.41	87.09	87.92
LED	68.92	**73.66**	69.22	72.32	73.65	73.56	73.62	73.43
RandomRBF	93.28	**93.48**	86.11	91.64	93.73	86.38	91.91	89.58
Average	**80.40**	78.27	77.85	78.42	78.03	69.13	76.70	72.36

Table 19
Accuracies of IncA-DES and methods of the state of the art with delayed and partial labels (Delay = 200).

Dataset	IncA-DES	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE
Ozone	91.94	**93.96**	93.95	93.40	**93.96**	**93.96**	93.87	**93.96**
Gas Sensor	56.06	51.39	**57.22**	51.85	48.52	55.50	50.75	52.82
Adult	82.91	82.89	82.47	82.84	83.89	84.13	84.03	**84.15**
Rialto	34.07	**35.05**	33.01	35.04	27.41	27.46	24.38	28.16
Keystroke	**94.42**	89.59	74.01	69.80	88.36	67.64	77.14	49.14
Insects-AB	**72.10**	69.94	67.99	70.52	70.65	54.53	65.69	61.63
Insects-GB	**74.62**	73.04	69.98	70.89	72.93	55.31	67.30	59.48
Insects-IB	60.22	**64.30**	59.46	62.62	64.07	49.14	58.87	55.14
Yeast	47.27	**47.70**	45.97	31.20	35.44	45.79	43.61	35.12
Asfault	**82.72**	80.70	78.87	80.17	82.55	68.13	76.01	66.51
Covertype	88.88	**92.10**	91.48	91.32	91.68	80.07	88.53	84.94

(*continued on next page*)

Table 19 (*continued*).

Dataset	IncA-DES	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE
Pen Digits	**91.04**	87.88	87.64	79.57	86.84	79.50	83.43	79.17
Dry Bean	86.59	88.02	85.24	87.39	86.15	**88.03**	87.20	84.70
Rice	88.66	**89.89**	85.23	88.27	89.48	88.82	88.35	80.16
Letters	72.08	30.43	**73.30**	49.73	43.90	57.34	56.57	57.89
SineRec	91.27	90.28	87.12	**96.91**	93.78	55.31	89.56	83.79
SeaRec	86.19	85.99	86.18	**88.51**	86.69	84.24	86.21	85.85
StaggerRec	**98.67**	98.15	96.31	98.26	98.37	71.24	97.52	94.59
Agrawal	81.44	78.08	77.31	**92.49**	76.85	61.30	77.18	80.82
Hyperplane	86.73	87.30	89.15	83.84	86.04	87.49	87.08	**87.86**
LED	68.91	**73.64**	69.28	72.58	73.61	73.58	73.61	73.36
RandomRBF	93.23	93.46	86.10	92.25	**93.72**	86.25	91.98	89.52
Average	**78.64**	76.54	76.24	75.88	76.13	68.85	74.95	71.31

Table 20
Accuracies of IncA-DES and methods of the state of the art with delayed and partial labels (Delay = 1000).

Dataset	IncA-DES	ARE	Dynse	DynEd	ARF	OzaBag	LevBag	OAUE
Ozone	92.14	93.63	**93.99**	92.65	93.83	93.96	93.83	93.96
Gas Sensor	39.35	41.15	46.98	36.52	39.81	**51.77**	44.30	48.36
Adult	83.01	82.93	82.50	82.34	83.81	**84.05**	84.04	84.03
Rialto	25.20	**31.67**	23.84	28.63	19.16	26.71	18.50	26.90
Keystroke	**85.26**	63.59	48.45	25.00	57.79	57.14	51.36	25.00
Insects-AB	**63.50**	61.93	60.12	62.00	61.84	53.18	59.02	56.50
Insects-GB	**67.36**	66.13	62.28	64.23	65.80	53.17	61.61	57.77
Insects-IB	58.96	**63.14**	58.48	60.47	62.89	48.21	57.79	53.87
Yeast	**47.11**	46.94	46.29	17.00	35.83	46.73	46.18	30.61
Asfault	**73.86**	71.34	69.30	61.66	73.69	60.71	67.38	50.77
Covertype	82.76	**86.56**	85.45	83.88	86.10	77.06	84.12	81.39
Pen Digits	**84.48**	81.84	81.21	73.27	79.80	74.04	77.74	73.03
Dry Bean	82.90	82.80	83.53	72.52	82.38	**83.78**	83.60	79.89
Rice	80.13	81.30	80.64	76.24	79.12	**82.51**	80.96	68.88
Letters	68.45	29.35	**70.30**	47.94	42.02	55.34	54.04	55.49
SineRec	84.97	83.85	81.17	**88.78**	87.11	51.63	82.96	77.30
SeaRec	85.43	85.29	85.55	**87.69**	85.89	84.14	85.46	85.20
StaggerRec	**93.78**	93.26	91.42	93.27	93.48	70.41	92.63	89.68
Agrawal	78.67	75.47	74.94	**88.41**	74.48	60.75	74.60	77.81
Hyperplane	86.59	87.12	**88.94**	83.15	85.79	87.22	86.93	87.42
LED	68.85	**73.55**	69.17	72.13	73.39	73.51	73.48	72.86
RandomRBF	93.18	93.29	85.94	92.06	**93.54**	86.31	91.78	89.16
Average	**73.91**	71.64	71.39	67.72	70.80	66.47	70.56	66.63

Data availability

Data will be made available on request.

References

[1] J.a. Gama, I. Žliobaitundefined, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Comput. Surv. 46 (2014).

[2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, IEEE Trans. Knowl. Data Eng. 31 (2019) 2346–2363.

[3] F. Ceschin, M. Botacin, A. Bifet, B. Pfahringer, L.S. Oliveira, H.M. Gomes, A. Grégio, Machine learning (in) security: A stream of problems, Digit. Threat.: Res. Pr. (2020).

[4] M.A. Shyaa, N.F. Ibrahim, Z. Zainol, R. Abdullah, M. Anbar, L. Alzubaidi, Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems, Eng. Appl. Artif. Intell. 137 (2024) 109143.

[5] G. Ditzler, R. Polikar, Incremental learning of concept drift from streaming imbalanced data, IEEE Trans. Knowl. Data Eng. 25 (2012) 2283–2301.

[6] L.I. Kuncheva, Classifier ensembles for changing environments, in: F. Roli, J. Kittler, T. Windeatt (Eds.), Multiple Classifier Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 1–15.

[7] R.M.A. Mohammad, A lifelong spam emails classification model, Appl. Comput. Inform. 20 (2024) 35–54.

[8] M. Müller, M. Salathé, Addressing machine learning concept drift reveals declining vaccine sentiment during the covid-19 pandemic, 2020.

[9] H. Guo, S. Zhang, W. Wang, Selective ensemble-based online adaptive deep neural networks for streaming data with concept drift, Neural Netw. 142 (2021) 437–456.

[10] C. Yang, Y.M. Cheung, J. Ding, K.C. Tan, Concept drift-tolerant transfer learning in dynamic environments, IEEE Trans. Neural Netw. Learn. Syst. 33 (2022) 3857–3871.

[11] H.M. Gomes, A. Bifet, J. Read, J.P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, T. Abdessalem, Adaptive random forests for evolving data stream classification, Mach. Learn. 106 (2017) 1–27.

[12] J. Kozal, F. Guzy, M. Woźniak, Employing chunk size adaptation to overcome concept drift, 2021.

[13] L.P. Cavalheiro, A.De.Souza. Britto, J.P. Barddal, L. Heutte, Dynamically selected ensemble for data stream classification, in: 2021 International Joint Conference on Neural Networks, IJCNN, 2021, pp. 1–7.

[14] M. Han, X. Zhang, Z. Chen, H. Wu, M. Li, Dynamic ensemble selection classification algorithm based on window over imbalanced drift data stream, Knowl. Inf. Syst. 65 (2023) 1105–1128.

[15] S. Abadifard, S. Bakhshi, S. Gheibuni, F. Can, Dyned: Dynamic ensemble diversification in data stream classification, in: CIKM '23: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, Association for Computing Machinery, New York, NY, USA, 2023, pp. 3707–3711.

[16] P.R. Almeida, L.S. Oliveira, A.S. Britto, R. Sabourin, Adapting dynamic classifier selection for concept drift, Expert Syst. Appl. 104 (2018) 67–85.

[17] B. Jiao, Y. Guo, D. Gong, Q. Chen, Dynamic ensemble selection for imbalanced data streams with concept drift, IEEE Trans. Neural Netw. Learn. Syst. (2022) 1–14.

[18] B. Wei, J. Chen, L. Deng, Z. Mo, M. Jiang, F. Wang, Adaptive bagging-based dynamic ensemble selection in nonstationary environments, Expert Syst. Appl. 255 (2024) 124860.

[19] R. Elwell, R. Polikar, Incremental learning of concept drift in nonstationary environments, IEEE Trans. Neural Netw. 22 (2011) 1517–1531.

[20] R.M. Cruz, R. Sabourin, G.D. Cavalcanti, Dynamic classifier selection: Recent advances and perspectives, Inf. Fusion 41 (2018) 195–216.

[21] R.M.O. Cruz, H.H. Zakane, R. Sabourin, G.D.C. Cavalcanti, Dynamic ensemble selection vs k-nn: Why and when dynamic selection obtains higher classification performance? in: 2017 Seventh International Conference on Image Processing Theory, Tools and Applications, IPTA, 2017, pp. 1–6, http://dx.doi.org/10.1109/IPTA.2017.8310100.

[22] D.N. Assis, F. Enembreck, J.P. Barddal, Mass-based short term selection of classifiers in data streams, in: 2023 International Joint Conference on Neural Networks, IJCNN, 2023, pp. 1–8.

[23] F. Bayram, B.S. Ahmed, A. Kassler, From concept drift to model degradation: An overview on performance-aware drift detectors, Knowl.-Based Syst. 245 (2022) 108632.

[24] F. Hinder, V. Vaquet, B. Hammer, One or two things we know about concept drift—a survey on monitoring in evolving environments. part a: detecting concept drift, Front. Artif. Intell. 7 (2024).

[25] J.Z. Kolter, M.A. Maloof, Dynamic weighted majority: An ensemble method for drifting concepts, J. Mach. Learn. Res. 8 (2007) 2755–2790.

[26] V. Agate, S. Drago, P. Ferraro, G.L. Re, Anomaly detection for reoccurring concept drift in smart environments, in: 2022 18th International Conference on Mobility, Sensing and Networking, MSN, 2022, pp. 113–120.

[27] G. Forman, Tackling concept drift by temporal inductive transfer, in: Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, Association for Computing Machinery, New York, NY, USA, 2006, pp. 252–259.

[28] L.L. Minku, A.P. White, X. Yao, The impact of diversity on online ensemble learning in the presence of concept drift, IEEE Trans. Knowl. Data Eng. 22 (2010) 730–742.

[29] F. Hinder, V. Vaquet, B. Hammer, One or two things we know about concept drift - a survey on monitoring in evolving environments. part b: Locating and explaining concept drift, Front. Artif. Intell. 7 (2024).

[30] W. Liu, H. Zhang, Z. Ding, Q. Liu, C. Zhu, A comprehensive active learning method for multiclass imbalanced data streams with concept drift, Knowl.-Based Syst. 215 (2021) 106778.

[31] A.S. Britto, R. Sabourin, L.E. Oliveira, Dynamic selection of classifiers—a comprehensive review, Pattern Recognit. 47 (2014) 3665–3680.

[32] N. Oza, Online bagging and boosting, in: 2005 IEEE International Conference on Systems, Man and Cybernetics, 2005, pp. 2340–2345 3.

[33] R. Davtalab, R.M. Cruz, R. Sabourin, A scalable dynamic ensemble selection using fuzzy hyperboxes, Inf. Fusion 102 (2024) 102036.

[34] M.A. Souza, R. Sabourin, G.D. Cavalcanti, R.M. Cruz, A dynamic multiple classifier system using graph neural network for high dimensional overlapped data, Inf. Fusion 103 (2024) 102145.

[35] R.M. Cruz, R. Sabourin, G.D. Cavalcanti, Meta-des.oracle: Meta-learning and feature selection for dynamic ensemble selection, Inf. Fusion 38 (2017) 84–103.

[36] J. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with drift detection, in: A.L.C. Bazzan, S. Labidi (Eds.), Advances in Artificial Intelligence – SBIA 2004, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 286–295.

[37] R.S. Barros, D.R. Cabral, P.M. Gonçalves, S.G. Santos, Rddm: Reactive drift detection method, Expert Syst. Appl. 90 (2017) 344–355.

[38] P. Domingos, G. Hulten, Mining high-speed data streams, Assoc. Comput. Mach. 7 (2000) 1–80.

[39] J.L. Bentley, Multidimensional binary search trees used for associative searching, Commun. ACM 18 (1975) 509–517.

[40] X. Wu, V. Kumar, J.Ross. Quinlan, J. Ghosh, Q. Yang, H. Motoda, G.J. McLachlan, A. Ng, B. Liu, P.S. Yu, Z.H. Zhou, M. Steinbach, D.J. Hand, D. Steinberg, Top 10 algorithms in data mining, Knowl. Inf. Syst. 14 (2008) 1–37.

[41] E.V.L. Barboza, P.R.L. de Almeida, A. de Souza Britto, R.M.O. Cruz, Distance functions and normalization under stream scenarios, in: 2023 International Joint Conference on Neural Networks, IJCNN, 2023, pp. 1–8.

[42] V. Perlibakas, Distance measures for pca-based face recognition, Pattern Recognit. Lett. 25 (2004) 711–724.

[43] M. Muja, D.G. Lowe, Fast approximate nearest neighbors with automatic algorithm configuration, in: International Conference on Computer Vision Theory and Applications, 2009, pp. 331–340.

[44] J. Jo, J. Seo, J.D. Fekete, A progressive k-d tree for approximate k-nearest neighbors, in: 2017 IEEE Workshop on Data Systems for Interactive Analysis, DSIA, 2017, pp. 1–5.

[45] A. Drozdek, Data Structures and Algorithms in C++, fourth ed., International Thomson Publishing, 2016.

[46] Y. Cai, W. Xu, F. Zhang, Ikd-tree: An incremental k-d tree for robotic applications, 2021, arXiv:2102.10808.

[47] Y. Chen, L. Zhou, Y. Tang, J.P. Singh, N. Bouguila, C. Wang, H. Wang, J. Du, Fast neighbor search by using revised k-d tree, Inform. Sci. 472 (2019) 145–162.

[48] G. Widmer, Combining robustness and flexibility in learning drifting concepts, in: European Conference on Artificial Intelligence, 1994, pp. 468–472.

[49] B. Pfahringer, G. Holmes, R. Kirkby, New options for hoeffding trees, in: M.A. Orgun, J. Thornton (Eds.), AI 2007: Advances in Artificial Intelligence, Springer Berlin Heidelberg, Berlin, Heidelberg, 2007, pp. 90–99.

[50] H. Yu, Q. Zhang, T. Liu, J. Lu, Y. Wen, G. Zhang, Meta-add: A meta-learning based pre-trained model for concept drift active detection, Inform. Sci. 608 (2022) 996–1009.

[51] E.B. Gulcan, F. Can, Unsupervised concept drift detection for multi-label data streams, Artif. Intell. Rev. 56 (2023) 2401–2434.

[52] V. Cerqueira, H.M. Gomes, A. Bifet, L. Torgo, Studd: a student–teacher method for unsupervised concept drift detection, Mach. Learn. 112 (2023) 4351–4378.

[53] M. Han, Z. Chen, M. Li, H. Wu, X. Zhang, A survey of active and passive concept drift handling methods, Comput. Intell. 38 (2022) 1492–1535.

[54] E.S. Page, Continuous inspection schemes, Biometrika 41 (1954) 100–115.

[55] M. Baena-García, J. Campo-Ávila, R. Fidalgo-Merino, A. Bifet, R. Gavald, R. Morales-Bueno, Early drift detection method, in: Proc. 4th Int. Workshop Knowledge Discovery from Data Streams, 2006.

[56] K. Nishida, K. Yamauchi, Detecting concept drift using statistical testing, in: V. Corruble, M. Takeda, E. Suzuki (Eds.), Discovery Science, Springer Berlin Heidelberg, Berlin, Heidelberg, 2007, pp. 264–269.

[57] I. Frías-Blanco, J.d. Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, Y. Caballero-Mota, Online and non-parametric drift detection methods based on hoeffding's bounds, IEEE Trans. Knowl. Data Eng. 27 (2015) 810–823.

[58] W. Hoeffding, Probability inequalities for sums of bounded random variables, J. Amer. Statist. Assoc. 58 (1963) 13–30.

[59] C. McDiarmid, On the Method of Bounded Differences, in: London Mathematical Society Lecture Note Series, Cambridge University Press, 1989.

[60] M.M.W. Yan, Accurate detecting concept drift in evolving data streams, ICT Express 6 (2020) 332–338.

[61] G.J. Ross, N.M. Adams, D.K. Tasoulis, D.J. Hand, Exponentially weighted moving average charts for detecting concept drift, Pattern Recognit. Lett. 33 (2012) 191–198.

[62] A. Bifet, R. Gavaldà, Learning from time-changing data with adaptive windowing, in: SDM, 2007, pp. 443–448.

[63] A. Pesaranghader, H.L. Viktor, Fast hoeffding drift detection method for evolving data streams, in: P. Frasconi, N. Landwehr, G. Manco, J. Vreeken (Eds.), Machine Learning and Knowledge Discovery in Databases, Springer International Publishing, Cham, 2016, pp. 96–111.

[64] A. Pesaranghader, H. Viktor, E. Paquet, Reservoir of diverse adaptive learners and stacking fast hoeffding drift detection methods for evolving data streams, Mach. Learn. 107 (2018) 1711–1743.

[65] S. Sakthithasan, R. Pears, Y.S. Koh, One pass concept change detection for data streams, in: J. Pei, V.S. Tseng, L. Cao, H. Motoda, G. Xu (Eds.), Advances in Knowledge Discovery and Data Mining, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 461–472.

[66] T. Peel, S. Anthoine, L. Ralaivola, Empirical bernstein inequalities for u-statistics, in: J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, A. Culotta (Eds.), Advances in Neural Information Processing Systems, Curran Associates, Inc., 2010, pp. 1903–1911.

[67] R. Pears, S. Sakthithasan, Y.S. Koh, Detecting concept change in dynamic data streams, Mach. Learn. 97 (2014) 259–293.

[68] L. Baier, T. Schlör, J. Schöffer, N. Kühl, Detecting concept drift with neural network model uncertainty. CoRR abs/2107.01873, 2021, arXiv:2107.01873.

[69] L. Breiman, Bagging predictors, Mach. Learn. 24 (1996) 123–140.

[70] Y. Freund, Boosting a weak learning algorithm by majority, Inform. and Comput. 121 (1995) 256–285.

[71] A. Bifet, G. Holmes, B. Pfahringer, Leveraging bagging for evolving data streams, in: J.L. Balcázar, F. Bonchi, A. Gionis, M. Sebag (Eds.), Machine Learning and Knowledge Discovery in Databases, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 135–150.

[72] L.L. Minku, X. Yao, Ddd: A new ensemble approach for dealing with concept drift, IEEE Trans. Knowl. Data Eng. 24 (2012) 619–633, http://dx.doi.org/10.1109/TKDE.2011.58.

[73] W.N. Street, Y. Kim, A streaming ensemble algorithm (sea) for large-scale classification, in: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA, 2001, pp. 377–382.

[74] H. Wang, W. Fan, P.S. Yu, J. Han, Mining concept-drifting data streams using ensemble classifiers, in: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA, 2003, pp. 226–235.

[75] D. Brzeziński, J. Stefanowski, Accuracy updated ensemble for data streams with concept drift, in: E. Corchado, M. Kurzyński, M. Woźniak (Eds.), Hybrid Artificial Intelligent Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 155–163.

[76] D. Brzezinski, J. Stefanowski, Combining block-based and online methods in learning ensembles from concept drifting data streams, Inform. Sci. 265 (2014) 50–67.

[77] A. Feitosa Neto, A.M. Canuto, Eocd: An ensemble optimization approach for concept drift applications, Inform. Sci. 561 (2021) 81–100.

[78] N. Liu, J. Zhao, Streaming data classification based on hierarchical concept drift and online ensemble, IEEE Access 11 (2023) 126040–126051, http://dx.doi.org/10.1109/ACCESS.2023.3327637.

[79] Y. Chen, X. Yang, H.L. Dai, Cost-sensitive continuous ensemble kernel learning for imbalanced data streams with concept drift, Knowl.-Based Syst. 284 (2024) 111272.

[80] X. Zhu, X. Wu, Y. Yang, Dynamic classifier selection for effective mining from noisy data streams, in: Fourth IEEE International Conference on Data Mining, ICDM'04, 2004, pp. 305–312.

[81] P. Zyblewski, R. Sabourin, M. Woźniak, Preprocessed dynamic classifier ensemble selection for highly imbalanced drifted data streams, Inf. Fusion 66 (2021) 138–154.
[82] A.M. Paim, F. Enembreck, Adaptive regularized ensemble for evolving data stream classification, Pattern Recognit. Lett. 180 (2024) 55–61.
[83] A.M. Paim, F. Enembreck, Adaptive random tree ensemble for evolving data stream classification, Knowl.-Based Syst. 309 (2025) 112830.
[84] X. Zhu, X. Wu, Y. Yang, Effective classification of noisy data streams with attribute-oriented dynamic classifier selection, Knowl. Inf. Syst. 9 (2006) 339–363.
[85] V.M.A. Souza, D.M. dos Reis, A.G. Maletzke, G.E.A.P.A. Batista, Challenges in benchmarking stream learning algorithms with real-world data, Data Min. Knowl. Discov. 34 (2020) 1805–1858.
[86] A. Bifet, G. Holmes, R. Kirkby, B. Pfahringer, MOA: massive online analysis, J. Mach. Learn. Res. 11 (2010) 1601–1604.
[87] M. Kelly, R. Longjohn, K. Nottingham, The uci machine learning repository. URL: https://archive.ics.uci.edu.
[88] M. Harries, N.S. Wales, et al., Splice-2 Comparative Evaluation: Electricity Pricing, University of New South Wales, School of Computer Science and Engineering, 1999.
[89] V. Rajkovic, Nursery, 1997, UCI Machine Learning Repository.
[90] K. Zhang, W. Fan, X. Yuan, Ozone level detection, 2008, http://dx.doi.org/10.24432/C5NG6W, UCI Machine Learning Repository.
[91] A. Vergara, Gas sensor array drift dataset, 2012, http://dx.doi.org/10.24432/C5RP6W, UCI Machine Learning Repository.
[92] B. Becker, R. Kohavi, Adult, 1996, UCI Machine Learning Repository.
[93] V. Losing, B. Hammer, H. Wersing, Knn classifier with self adjusting memory for heterogeneous concept drift, in: 2016 IEEE 16th International Conference on Data Mining, ICDM, 2016, pp. 291–300.
[94] V.M.A. Souza, D.F. Silva, J. Gama, G.E.A.P.A. Batista, Data stream classification guided by clustering on nonstationary environments and extreme verification latency, in: Proceedings of the 2015 SIAM International Conference on Data Mining, SDM, 2015, pp. 873–881.
[95] J. Blackard, Covertype, 1998, http://dx.doi.org/10.24432/C50K5N, UCI Machine Learning Repository.
[96] K. Nakai, Yeast, 1991, http://dx.doi.org/10.24432/C5KG68, UCI Machine Learning Repository.
[97] V.M. Souza, Asphalt pavement classification using smartphone accelerometer and complexity invariant distance, Eng. Appl. Artif. Intell. 74 (2018) 198–211.
[98] D. Slate, Letter recognition, 1991, http://dx.doi.org/10.24432/C5ZP40, UCI Machine Learning Repository.
[99] E. Alpaydin, C. Kaynak, Optical recognition of handwritten digits, 1998, http://dx.doi.org/10.24432/C50P49, UCI Machine Learning Repository.
[100] M. Koklu, I.A. Özkan, Multiclass classification of dry beans using computer vision and machine learning techniques, Comput. Electron. Agric. 174 (2020) 105507.
[101] I. Cinar, M. Koklu, Classification of rice varieties using artificial intelligence methods, Int. J. Intell. Syst. Appl. Eng. (2019).
[102] J.C. Schlimmer, R.H. Granger, Incremental learning from noisy data, Mach. Learn. 1 (1986) 317–354.
[103] R. Agrawal, T. Imielinski, A. Swami, Database mining: a performance perspective, IEEE Trans. Knowl. Data Eng. 5 (1993) 914–925.
[104] G. Hulten, L. Spencer, P. Domingos, Mining time-changing data streams, in: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA, 2001, pp. 97–106.
[105] A. Bifet, G. Holmes, B. Pfahringer, R. Kirkby, R. Gavaldà, New ensemble methods for evolving data streams, in: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009.
[106] J. Montiel, J. Read, A. Bifet, T. Abdessalem, Scikit-multiflow: A multi-output streaming framework, J. Mach. Learn. Res. 19 (2018) 1–5.
[107] A.H. Ko, R. Sabourin, A.S. Britto Jr., From dynamic classifier selection to dynamic ensemble selection, Pattern Recognit. 41 (2008) 1718–1731.